Implementation of Game of Blocks world with AI – Dueler

Abstract

The "Blocks World" is a classic scenario for artificial intelligence (AI) algorithm verification and interactive entertainment (Nilsson, 1980). Traditional AI for block stacking games mostly relies on fixed rules to complete tasks, lacking adaptive decision-making capabilities, especially in multi-player adversarial scenarios with shared space. To address this limitation, this project designs and implements Blocks World with AI-Dueler, a 3D block stacking game integrated with multi-mode gameplay and intelligent AI opponents. The game supports three core modes: single-player training mode, multi-player racing mode (host creation & game hall joining), and shared-space adversarial mode.

This project proposes a two-stage training + dynamic camouflage architecture to solve the core challenges of AI in shared-space mode, including player target inference, strategic decision-making, and behavior concealment. For the racing mode, a heuristic model is adopted to achieve efficient block stacking without pre-training. For the adversarial mode, the AI capability is trained in two stages: the first stage trains a target prediction model based on PyTorch, and the second stage completes adversarial strategy training via the PPO algorithm from Ray RLlib, enabling the AI to dynamically choose between interfering with the player's key blocks and focusing on its own stacking goal. A phased camouflage strategy is introduced to hide the AI's real target from players, addressing the limitation of single and predictable behavior of heuristic models. Technically, the front-end is built with Vue2 and Three.js to implement a 3D interactive interface, and the back-end is developed with Python and Flask to complete online communication, AI model invocation, and victory judgment.

Test results show that the AI in racing mode reaches the operational efficiency of ordinary human players, while the AI in adversarial mode can accurately predict the player's target and significantly enhance the game's challenge. This project breaks through the limitations of traditional fixed-rule AI, constructs an intelligent agent framework of "target prediction - strategic confrontation - dynamic camouflage", and provides a feasible solution for the design of intelligent agents in block-based competitive games, with both academic reference value and practical application significance.

Statement of Ethical Compliance

Data Category: B

Participant category: 2

I confirm that I have read the ethical guidelines and will follow them during this project.

Table of content

Abstract 2

Statement of Ethical Compliance 3

Table of content 4

1 Introduction & Background 6

2 Design 8

2.1 Front-end application layer 8

2.2 Backend service layer 13

2.3 AI model layer 18

3 Implementation 23

3.1 Front-end development 23

3.2 Backend development 30

3.3 AI model development 35

4 Testing & Evaluation 48

4.1 Usability testing 49

4.2 Model usability testing 49

4.3 User feedback 52

5 Project Ethics 55

6 Conclusion and further work 55

6.1 Conclusion 55

6.2 Future Work 56

7 BCS Criteria and Self-Reflection 57

7.1 Compliance with BCS Project Criteria 57

7.2 Self-Evaluation 58

References 60

Appendix 64

Appendix 1. Storyboard 64

Appendix 2. Non-Interference Data Generation Output 69

Appendix 3. Reward weights 70

Appendix 4. Test based on storyboard 71

Appendix 5. Test of accuracy of predicted target 79

Appendix 6. Participant Consent Form 80
Appendix 7. Feedback form 82
Appendix 8. Project structure 83

1 Introduction & Background

The Blocks World is a classic research paradigm in the field of artificial intelligence, widely used to verify the effectiveness of search algorithms and planning strategies (Nilsson, 1980; Slaney & Thiébaux, 2001). Traditional AI systems for Blocks World games are mostly built on fixed rules, which can only complete path optimization in single-player scenarios. However, in multi-player adversarial scenarios with shared block resources, these systems lack the ability to perceive opponents' intentions and dynamically adjust strategies, resulting in rigid behavior and poor playability.

With the development of deep reinforcement learning, algorithms such as AlphaZero, MADDPG and PPO have achieved a leap from rule-following to autonomous learning, providing technical support for solving multi-agent adversarial problems (Silver et al., 2018; Lowe et al., 2017; Schulman et al., 2017). While existing research has made breakthroughs in adversarial scenarios such as chess and e-sports (Mnih et al., 2015), there are still research gaps in the exploration of key technologies such as target reasoning and strategy camouflage for Blocks World in shared-space adversarial mode. This gap provides the core research motivation and implementation space for this project.

This project aims to develop a 3D block stacking game with intelligent adversarial capabilities, upgrading the traditional Blocks World from a simple puzzle game to a strategic adversarial game by endowing the AI with intention perception, dynamic decision-making and behavior camouflage capabilities. Meanwhile, the project verifies the effectiveness of the combination of deep learning and reinforcement learning in intention reasoning scenarios through the observable shared-space features of Blocks World, providing a reference for the design of similar strategy games.

The main goal of this project is to complete the development of a full-featured Blocks World game with single-player, multi-player racing, shared-space adversarial and human-AI battle modes, and build an adaptive Dueler agent that can infer the opponent's target through operation analysis and make independent strategic decisions in shared-space mode. The specific requirements are divided into necessary requirements and expected requirements as follows:

The Necessary Requirements consist of:

- Implement 3D block visualization and complete basic game operation logic, supporting mouse

drag-and-drop interaction and real-time victory/defeat judgment;

- Build a Flask-based online battle system, supporting multi-player game creation, room joining and real-time game state synchronization;
- Implement the core adversarial capability of the Dueler agent, with its operational efficiency in racing mode not lower than that of ordinary human players.

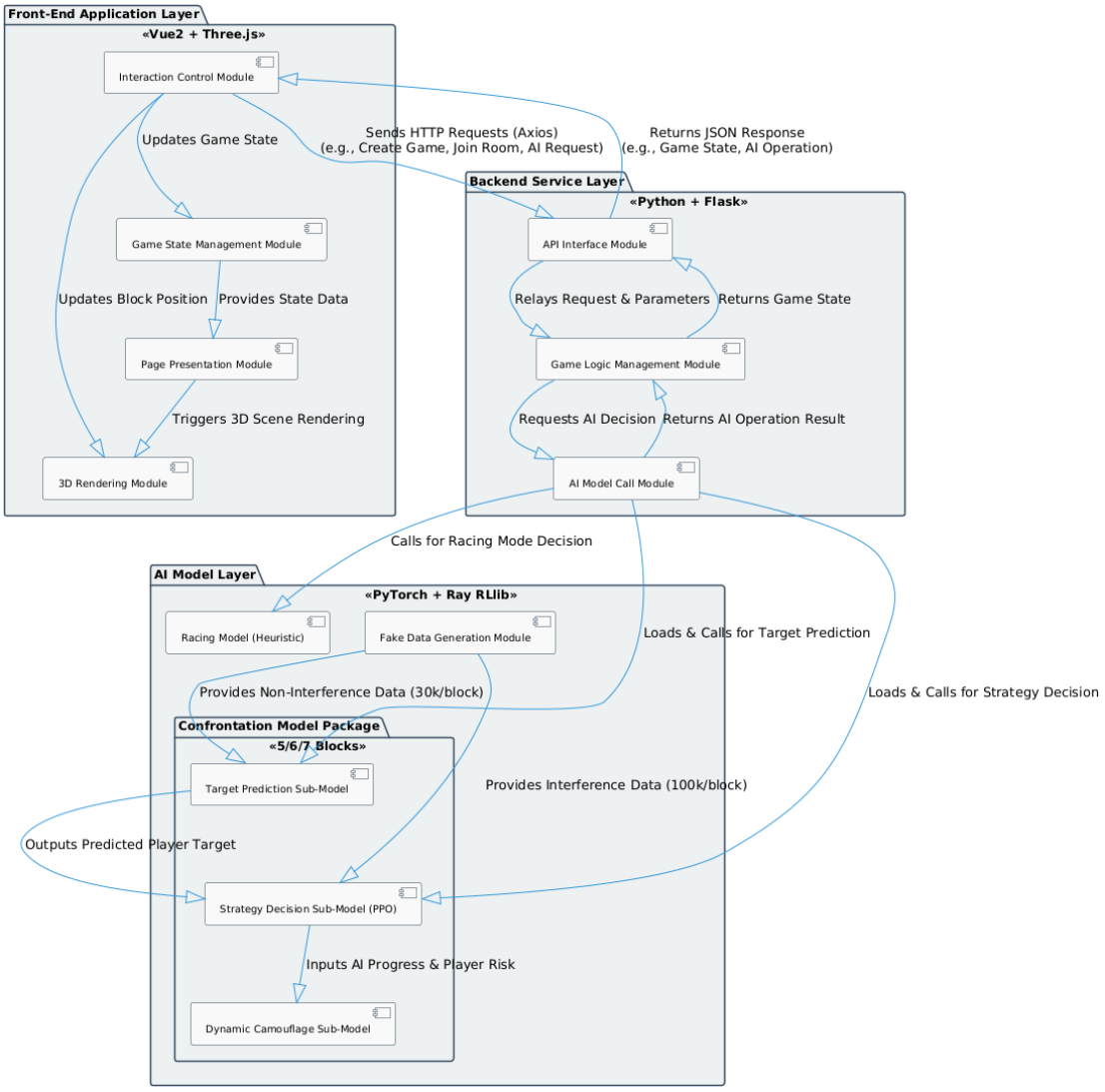
The Expected Requirements consist of:

- Realize dynamic adjustment of the strategy difficulty of the Dueler agent;
- Support customized configuration of the number of blocks and turn time;
- Add game sound effects, action animations and other interactive feedback elements.

To ensure the readability of the dissertation for the general computer science audience, the core terms and acronyms involved in this research are defined as follows:

- Dueler: The intelligent adversarial AI developed for this project includes three core functions: target prediction, strategy game and dynamic camouflage.
- Shared Space Mode: A turn-based adversarial mode where the player and the AI share the same set of cube resources and each side holds an undisclosed stack goal;
- PPO (Proximal Policy Optimization) : This project is a reinforcement learning algorithm used to train the Dueler agent confrontation strategy. The algorithm has excellent stability and sample utilization efficiency in multi-agent confrontation scenarios.

2 Design



The overall architecture of the system adopts a three-layer design of front-end application, back-end service and AI model layer. AI model layer". Each layer assumes independent responsibilities and implements functions through standardized interaction protocols. The overall organization follows the principle of "separation of concerns" to ensure that the front-end focuses on user interaction and 3D visualization, the back-end focuses on business logic and resource scheduling, and the AI model layer focuses on algorithm training and intelligent decision-making.

2.1 Front-end application layer

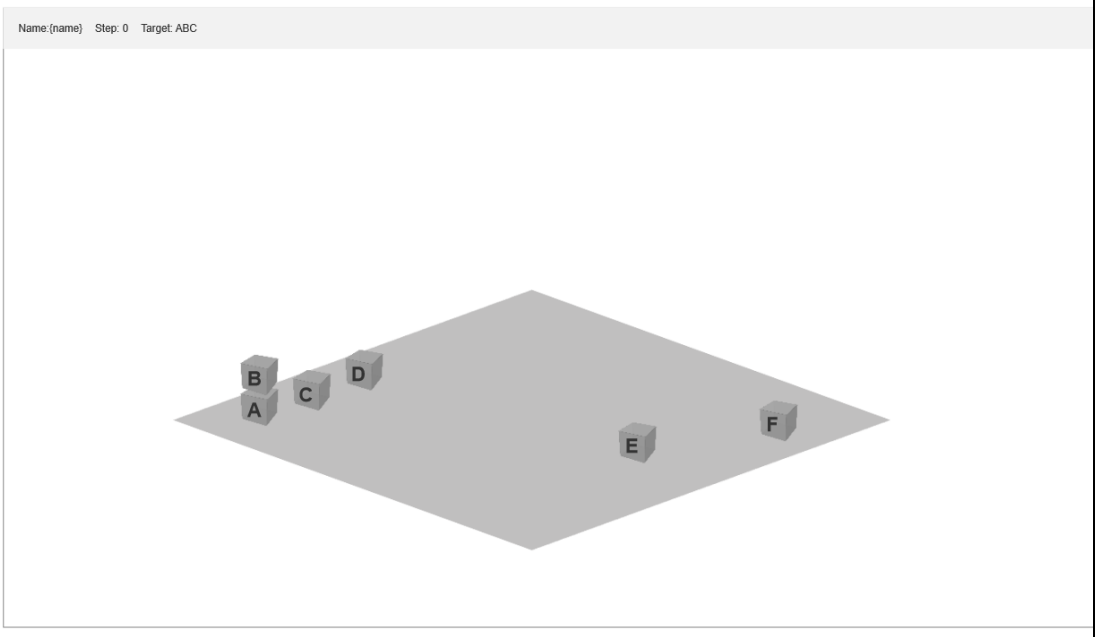
The frontend, which is responsible for user interaction, 3D scene rendering, and local logic processing, is deployed independently and only communicates with the backend via network

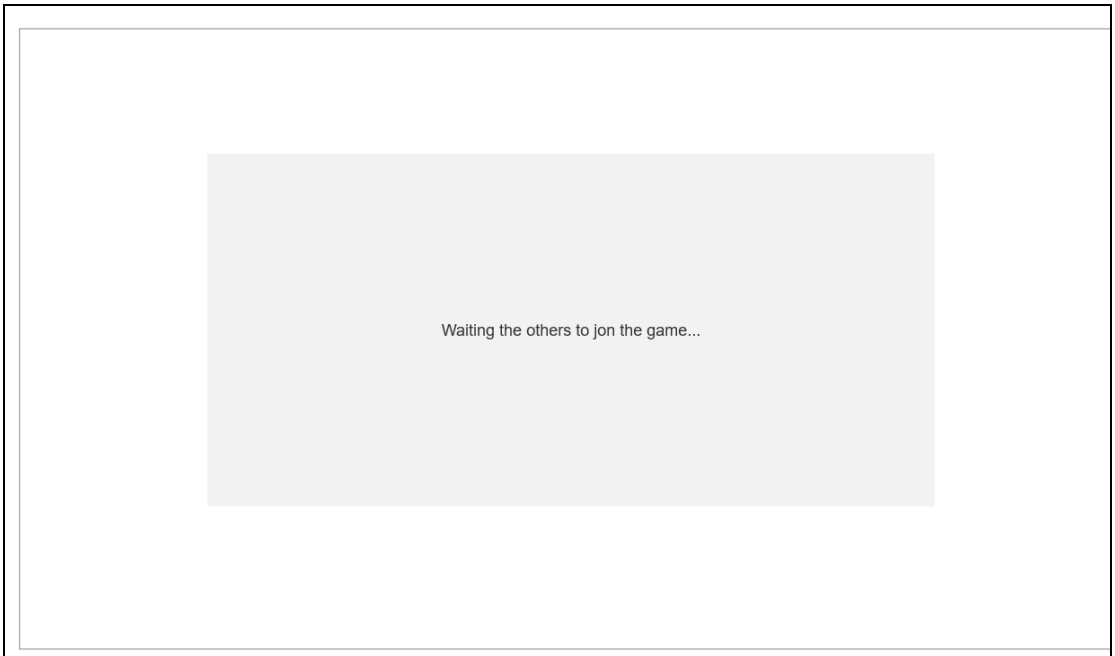
requests (e.g., multiplayer online, man-machine confrontation).

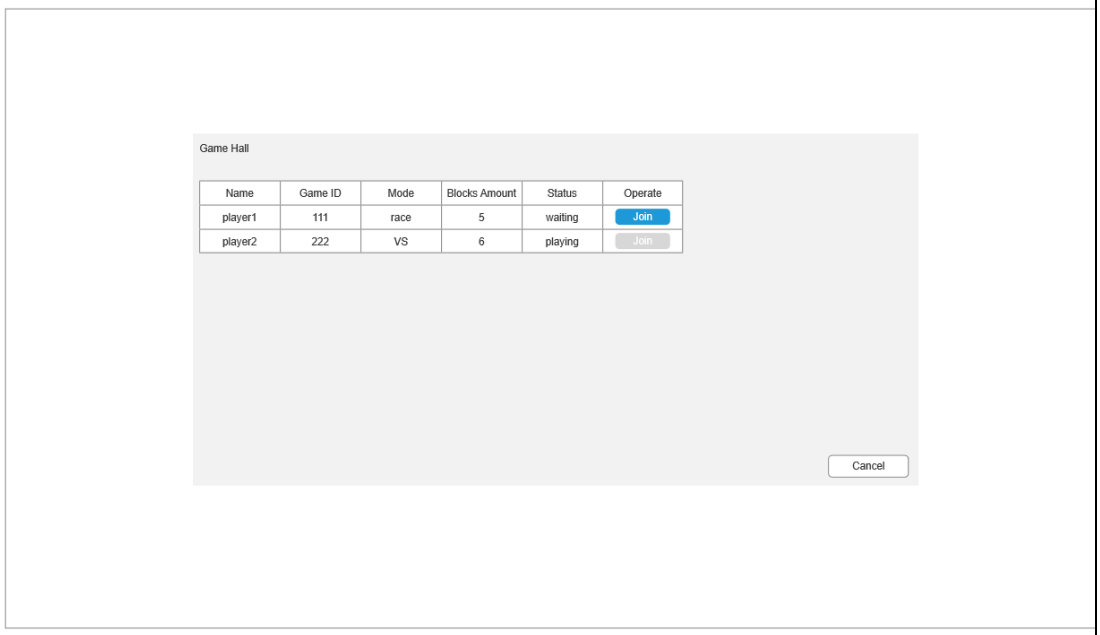
Model	Function	Key technologies/interaction logic
Display	Provide all functional pages of the game, including welcome page, game mode selection page, single player game page, multi-player racing page, shared space confrontation page, game hall, etc.	Vue2 (Vue.js, 2024) component development, through Vue Router to achieve page switching (Vue.js, 2024a); Element UI integrates components such as action guidance, pop-up prompts, etc. (Eleme, 2024)
3D rendering	Complete 3D visualization of blocks, game scenes, and operation feedback, including block rendering, identification (A/B/C), movement animation, and invalid operation prompts.	Three.js, build 3D scene (camera, light source, desktop), realize the block model loading and position update; Bind block and stack movement to mouse events.
Interactive control	Handles user actions (mode selection, block drag and drop, game creation/join), and manages local front-end logic.	Axios: Sends HTTP requests to the backend (e.g., create a multiplayer game, join a lobby, request an AI decision) (Axios, 2024)
Game state Management	Maintains the global state of the game, including game mode, number of blocks, player information, goal status (hidden in confrontation mode), and turn progress.	sessionStorage stores temporary state (such as the current player's name and number of blocks)

2.1.1 Sketchers

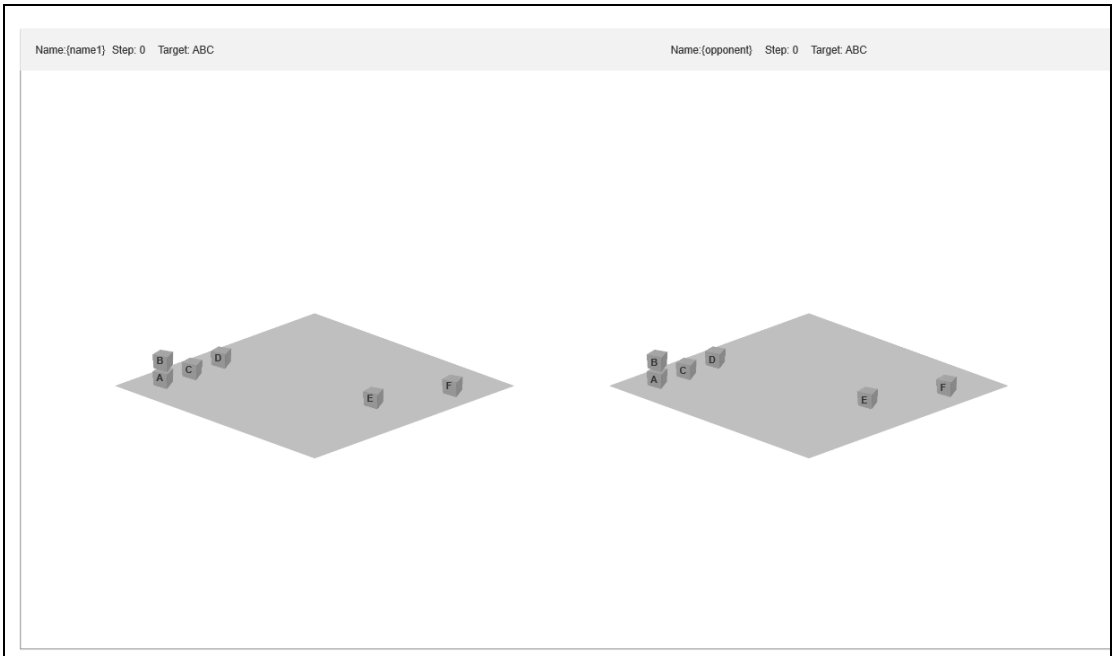
Welcome

Single Game

Game Hall

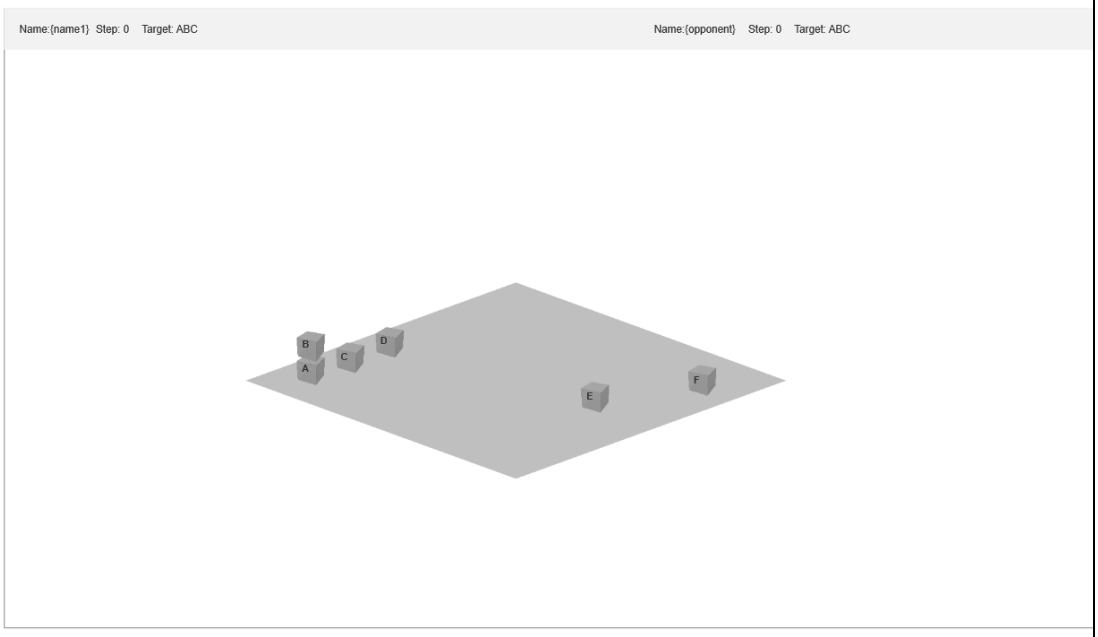

Waiting the others to jon the game...


Game Hall
Name
Game ID
Mode
Blocks Amount
Status
Operate
player1
111
race
5
waiting
Join
player2
222
VS
6
playing
Join
Cancel

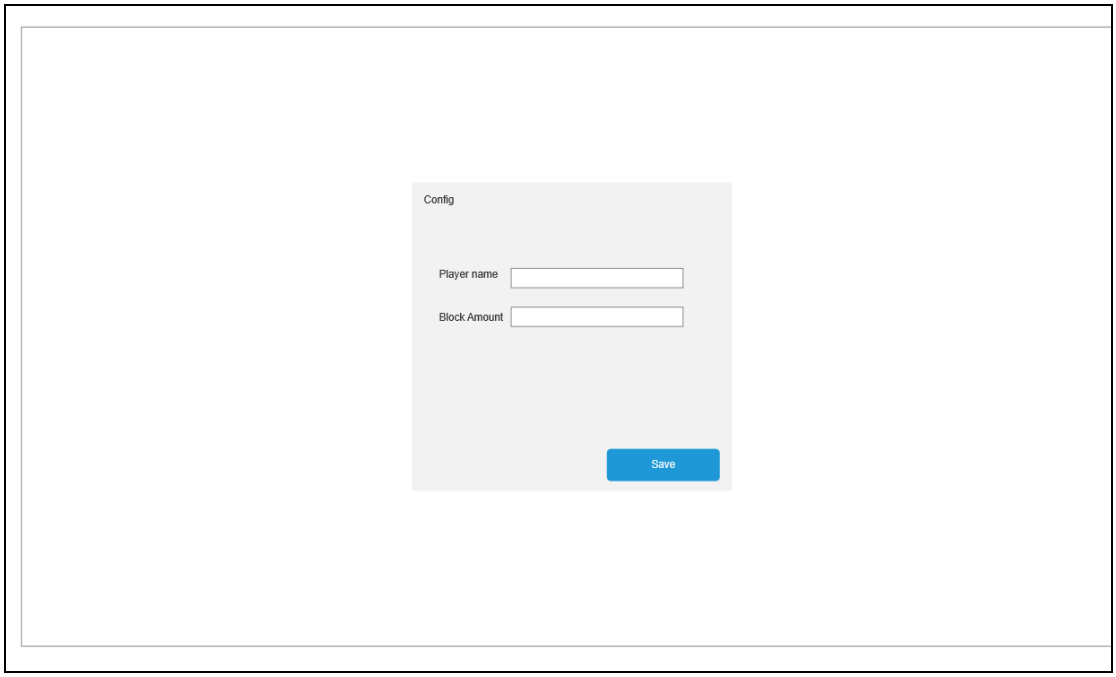
Race Mode



Confrontation mode



Config



2.1.2 Storyboard

This project designed the interaction process of four core gameplay: single-player training, multi-player racing, human-machine racing and human-machine confrontation. The single-player mode supported players to complete the block stacking training based on the target sequence. The multi-player racing mode adopts the online mechanism of homeowner creation-hall joining, and the dual-scene synchronous racing, the first to complete the target wins. Human-machine racing mode player and heuristic-based Dueler racing; the man-machine confrontation mode is turn-based shared space play, in which the player and the AI share block resources, each holding hidden targets, and complete the confrontation through stacking, camouflage and interference, and details could be found in appendix 1.

2.2 Backend service layer

The back-end service layer is built based on the design principles of modularity, low coupling, scalability, and fault tolerance first, which is suitable for the AI confrontation scenario of multi-party block mode (Pallets Projects, 2024).

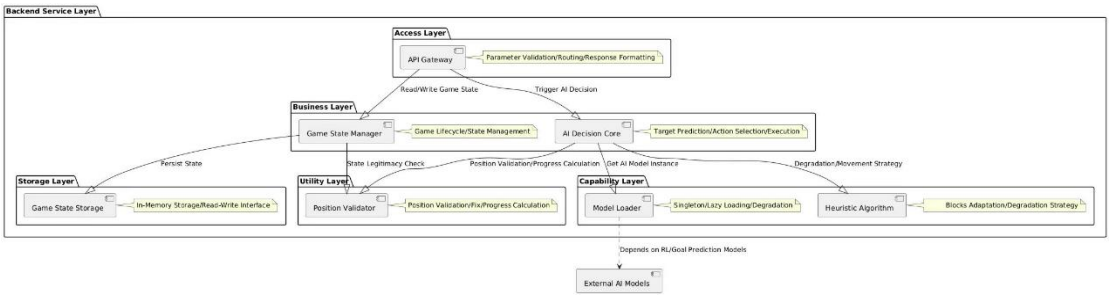
The "model loading, AI decision, game state management, interface interaction and tool verification" were divided into independent modules, and the modules communicated through

standardized interfaces to avoid strong dependencies. The RL model, the target prediction model, and the heuristic algorithm were hierarchically encapsulated, and the front end did not need to perceive the internal implementation of AI, but only called through a unified interface.

An adaptation layer driven by the number of blocks was designed to support the adaptive switching of different number of blocks, algorithms, and observation/action spaces.

When core dependencies (such as AI models) fail to load, they are automatically downgraded to heuristic strategies to ensure service availability. The game state verification and repair mechanism were designed to ensure the legitimacy of data such as block position and turn state. Unified RESTful interface style, request/response format, and status code to reduce the cost of front-end and back-end interaction (Fielding, 2000).

2.2.1 Module layering and responsibilities

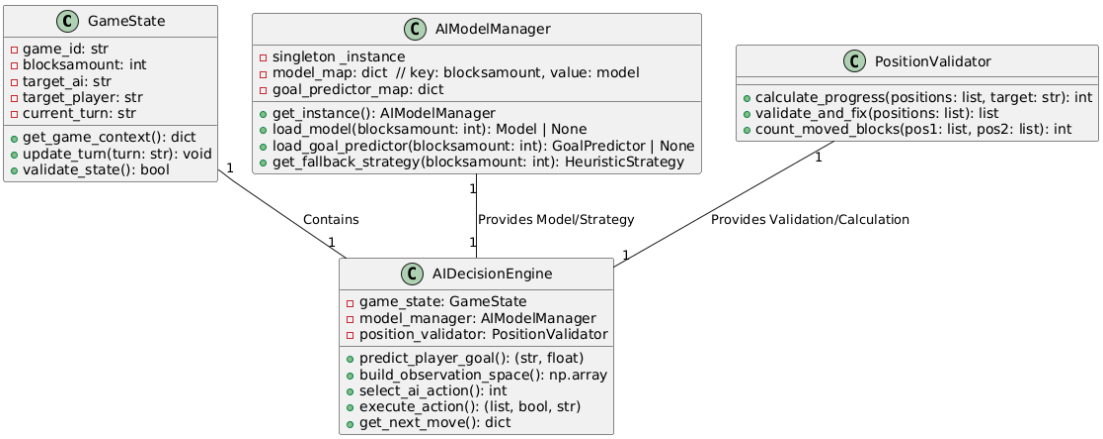


Module Layer	**Core Sub-Module**	**Design Responsibilities**	**Interaction Logic with Other Modules**
Access Layer	API Gateway	Receive front-end requests, parameter validation, route distribution, response formatting, version control	Receive front-end requests → Validate parameters → Route to corresponding business layer interfaces → Format business layer return results and respond
Business Layer	Game State Management	Maintain game lifecycle (creation/query/update), manage multi-mode game states (Dueler/VS AI)	Provide game state to AI Decision Core Module → Receive AI decision

			results → Update game state → Synchronize to API Gateway Module
	AI Decision	Target prediction, observation space construction, action selection, action execution (interference/movement/deception)	Obtain context from Game State Management Module → Call Model Loader Module to get AI models → Invoke Utility Module for position validation/fixing
Capability Layer	Model Loader	Singleton management of AI models (RL/goal prediction), lazy loading, failure degradation	Respond to model requests from AI Decision Core Module → Load models corresponding to the number of blocks → Return model instance/degradation flag
	Heuristic Algorithm	Encapsulate heuristic strategies for different block modes as a fallback for AI models	Receive action requests from AI Decision Core Module → Output block movement steps that meet the target
Utility Layer	Position Validation and Fixing	Verify the legitimacy of block positions, fix overlapping positions, calculate player/AI progress	Provide position validation for AI Decision Core Module → Offer progress calculation for

			Game State Management Module
Storage Layer	Game State Storage	In-memory persistence of core data (game ID, target sequence, number of blocks, turn state, etc.)	Provide read-write interfaces for Game State Management Module

2.2.2 Core class design



The GameState is the core container for game state.This module encapsulates attributes such as the game ID, the number of blocks, the target sequence of the AI and the player, and the current turn, and is responsible for controlling key states throughout the life of the game. The get_game_context, validate_state and other methods provided by the system not only provide a unified state data source for other modules of the system, but also effectively ensure the legality and validity of the game state.

AIModelManager is the resource scheduling center of AI models. This module manages AI models and goal predictors for different numbers of squares through model_map and goal_predictor_map. At the same time, relying on load_model, get_fallback_strategy and other methods, the model lazy loading and fault degradation mechanism are realized. This design not only avoids the problem of resource redundancy in the system startup phase, but also continuously ensures the availability of AI functions.

PositionValidator is the data verification and calculation tool of the system. calculate_progress, validate_and_fix and other methods are installed in the positionValidator, which allows AI decisions to have the ability to verify the validity of the square position. On the other hand, it can

accurately calculate the stacking progress of players and AI, which is the underlying support to ensure the compliance of game states and the effective behavior of AI.

AIDecisionEngine is the AI decision center of the whole system. This module integrates three core dependencies: GameState, AIModelManager and PositionValidator. Through the predict_player_goal, select_ai_action, get_next_move and other methods, the whole process decision of state acquisition, goal prediction and action generation is completed. It provides support for AI to achieve core capabilities such as interference, mobility, and camouflage.

The collaboration of these four core classes forms a closed loop of "state management - model scheduling - data validation - decision execution", providing backend logical support for the intelligent confrontation scenarios of block games.

2.2.3 Interface design

Unified use of JSON format, must include request_id, timestamp; the business parameter needs to include game_id (unique identifier of the game), and the multi-mode scene needs to pass blocksamount.

The game_id is in UUID format, position coordinates are nonnegative integers (Leach et al., 2005), and the target sequence length is the same as blocksamount.

```
{
"request_id": "xxx-xxx-xxx",
"code": 200,// status code
"message": "success", // description
"data": {}, // data
"timestamp": 1718xxxxxxx
}
```

Code	Meaning	Example
200-299	Success	Create a game
500-599	Service error	500 (AI decision failure),(model loading failure)

Based on the design in sketchers, the blocks are existed in a 3d space, which means the position of each block should be the Three-dimensional coordinates.

```
[
{"block": "A", "x": 10, "y": 10, "z": 0},
{"block": "B", "x": 11, "y": 10, "z": 1},
{"block": "C", "x": 10, "y": 10, "z": 2},
{"block": "D", "x": 8, "y": 8, "z": 0},
{"block": "E", "x": 8, "y": 8, "z": 1},
{"block": "F", "x": 8, "y": 8, "z": 2},
{"block": "G", "x": 8, "y": 8, "z": 3}
]
```

2.2.4 Data design

There is no database interaction in the current design, and all information is kept in python services. A global variable list is used to store the game rooms in hall, like Store publicly visible basic game room information (e.g., room ID, number of participants, game mode).

For the Dueler model, it stores the following information:

Name	Type	Description	Example
solution_steps	list[Step]	A complete sequence of steps for the AI to reach the goal	[{block:'A',...}], ...]
target	str	AI target block stacking order	"ABCDE"
blocksamount	int	Block amount	5

2.3 AI model layer

In VS mode, AI is required to implement a three-tier core logic of "Goal Inference - Strategy Decision-Making - Autonomous Stacking", neither players nor the AI can know each other's stacking goals. The AI first infers the player's core goal through their block movement behaviors, then dynamically decides to either "interfere with the player's stacking" or "focus on achieving its own goal". In the game logic, the core role of the interference strategy is to accurately interrupt

the core stacking path of the player, and the autonomous stacking module can efficiently complete its goal without relying on pre-training. Combining the characteristics of this kind of game theory with the practical requirements of system efficiency, this project designed a set of architecture system. The system is composed of Player Goal Prediction Model, Interference Strategy Decision-Making Model and Heuristic Stacking Algorithm.

2.3.1 Generation of training data

The core of the square stacking problem lies in the temporal association between "state-action-goal", rather than simple arrangement and combination. Therefore, when constructing a dataset, it is necessary to cover the state transition and behavior feedback information during the whole game.

The dataset is mainly divided into two categories, which are used for object prediction training and interference decision training, respectively. In the training phase related to target prediction, more than 10,000 data records are designed and generated, each record contains global configuration, timing state and target information (Korf, 1985; Slaney & Thiebaux, 2001).

Specific fields include:

- Total number of blocks (including blocks to be moved + a random number of distractor blocks)
- Target arrangement (randomly selected from permutations, e.g., ABCDFE)
- Initial positions (3D coordinates of all blocks)
- Step sequence (full positional state of all blocks at each step)
- Total number of steps

This dataset focuses on the target achievement process under player-non-interfering conditions, covering diverse initial layouts, distractor block counts, and target combinations. This part provides data support for training the mapping relationship between player behavior and target outcome.

To train the interference decision model, more than ten thousand data records are designed and generated, each record contains information such as player goals, timing actions and game results (Korf, 1985; Slaney & Thiebaux, 2001), which includes:

Number of core block (block to be moved)

- Player target permutation
- AI target permutation (different from the player’s target)

- Player progress before interference (e.g., 4–5 blocks stacked successfully)
- Step at which interference occurs
- Independent step counts for the player and the AI
- Initial positions
- Complete step sequence (including continuous states of player stacking and AI interference)
- Completion status for both sides (player incomplete, AI complete)

Data generation focuses on critical junctures (when the player is near goal achievement), simulating varied interference timings and action intensities. This data provides direct action-reward feedback to evaluate the AI's interference strategy.

2.3.2 Heuristic Stacking Algorithm (Autonomous Stacking Strategy)

AI autonomous stacking adopts a heuristic algorithm framework guided by physical constraints, which can efficiently complete the core goal of block stacking within physical constraints and limited steps.

Heuristic algorithms do not require large volumes of training data. Explicit heuristic rules can be designed based on the physical rules of stacking tasks, delivering better performance than reinforcement learning.

The algorithm draws on the heuristic search concept of the classic A* algorithm (Hart et al., 1968): heuristic evaluation criteria are designed using domain knowledge to replace global exhaustive search and improve efficiency. As proven in 'A formal basis for the heuristic determination of minimum cost paths', the A* algorithm is using a heuristic function to filter non-optimal paths — physical validity (support, collision-free, non-overlapped) serves as the primary heuristic evaluation metric. Actions that satisfy physical constraints are processed first to quickly locate feasible stacking sequences.

It defines the stack state evaluation function $f(s)$ to measure the physical legitimacy of the current state s and the matching degree with the target as the core basis for heuristic selection. Taking the 6 blocks as an example, the formular is:

$$f(s) = \alpha \cdot \sum_{i=1}^{6} I\big(pos_i(s) = pos_i(G_{AI})\big) + (1-\alpha) \cdot \frac{1}{6} \sum_{i=1}^{6} I(legal_i(s))$$

$pos_i(s)$ is position of block i in current state; $pos_i(G_{AI})$ is the target position of block i. $I()$ is an indicator function (1 if the condition is met, 0 otherwise).

$legal_i(s)$ is the physical validity of block i in the current state (collident-free, supported, not overridden).

α∈[0,1] is the weight coefficient (α=0.6 in this design, which gives priority to ensuring the target matching degree and takes into account the physical legality).

f(s)∈[0,1], that the larger the value, the closer to a "physically legal goal state".

The heuristic action takes "physical constraints first and approach the target step by step" as the rule, which means the squashed blocks are released first, and the unconstrained blocks are processed later. Blocks are processed from low to high target height, and the block movement is broken down into three stages of "rising to a safe height → moving horizontally to the target coordinate → falling to the support position" to avoid path collision (Hart et al., 1968).

If the target position is occupied, the occupied square is moved to the free position on the ground. The "minimum path length" is used as the heuristic criterion in the moving process.

2.3.3 Object Prediction Model (Supervised learning with PyTorch)

The essence of player target prediction is "3D coordinate sequence of high-dimensional time series blocks → target classification". Traditional machine learning algorithms (SVM, random forest) cannot effectively capture the long-distance dependence features of time series. Recurrent Neural Network (RNN) has the problem of gradient vanishing/exploding and low parallel computing efficiency (Vaswani et al., 2017).

With the self-attention mechanism, Transformer can directly model the dependence at any position in the time series, without recursive calculation, and has better parallelism (Lim et al., 2021; Zhang et al., 2024). Stacking multi-layer encoders can also strengthen the ability to capture long-term dependence. At the same time, PyTorch's dynamic computation graph features adapt to the dynamic state updates of the game (variable number of steps and blocks), so the "Transformer encoder + fully connected layer" architecture is selected (Lim et al., 2021; Zhang et al., 2024).

This design refers to the core conclusion of Attention Is All You Need (Vaswani et al., 2017). The Transformer self-attention mechanism has both better long dependence capture ability and parallel efficiency in time series modeling. AI intention prediction in square games (Zhang et al., 2024) also verified the effectiveness of this architecture in discrete state game goal prediction.

The model input is a time series excluding the last step goal state (to avoid data leakage) $X = \{P1$

, $P2, \ldots, PT\}$, in which T is the number of observation steps, $PT \in R^{N\times 3}$, the 3D coordinates of all squares at step t, and N is the maximum number of squares.

Output the probability distribution of a fixed number of class target arrangements, such as 30 instead of all classes, for accelerating training (Paszke et al., 2019).

In temporal feature extraction layer, the 3D coordinates are mapped to the high-dimensional feature space by MLP to improve the feature representation ability.

Secondly, the coding features of all blocks in a single step are aggregated to obtain the single-step timing features (Loshchilov & Hutter, 2019).

Multi-head self-attention maps features into Query, Key, and Value Spaces, and calculates multiple sets of attention weights in parallel, which captures the dependencies of different dimensions in time series data (Loshchilov & Hutter, 2017).

Each Transformer encoder layer contains "multi-head self-attention + residual connection + layer normalization + feedforward network + residual connection + layer normalization" to enhance the depth extraction ability of time series features (Loshchilov & Hutter, 2017).

Finally, it uses the cross-entropy loss to measure the difference between the predicted distribution and the true target.

$$Locss_{CE} = -\sum_{i=1}^{30} G_{true}(i) \cdot \log\left(\hat{G}(i)\right)$$

G true is the one-hot vector of the player's true goal. The optimizer chooses AdamW with weight decay to balance the convergence speed and parameter regularization. The learning rate is scheduled using cosine annealing (Paszke et al., 2019).

2.3.4 Interference Decision Model (Ray-based Reinforcement Learning)

Intervention strategy is a sequential decision-making problem in game scenarios, which requires continuous optimization through interaction with players. In this study, Ray framework is selected as the training framework, which natibly supports distributed reinforcement learning training and can realize parallel processing of large-scale intervention data, thus greatly improving the training efficiency of the model (Schulman et al., 2015; Schulman et al., 2017). At the algorithm level, this study uses the Proximal Policy Optimization (PPO) algorithm. Compared with the Deep Q-Network (DQN) algorithm (Mnih et al., 2015), the PPO algorithm constrained the step size of policy update by cutting the objective function, which could effectively avoid the problem of training instability

caused by excessive policy update.

The results of Yu et al. (2021) in The Surprising Effectiveness of PPO in Cooperative, Multi-Agent Games show that PPO algorithm can achieve high sample utilization efficiency and training stability in multi-agent confrontation scenarios. At the same time, the performance of the algorithm is also significantly better than other mainstream reinforcement learning algorithms in complex tasks such as Starcraft micromanipulation and soccer game (Moritz et al., 2018). Furthermore, the centralized training and decentralized execution paradigm proposed in “Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments" (Lowe et al., 2017) provides theoretical support for the model, ensuring that the AI can make globally optimal intervention decisions under local observations.

The core of PPO lies in trust region policy optimization. The policy is updated by maximizing the clipped objective function, ensuring that updates stay within a trustworthy region (Liang et al., 2018).

The clipped objective function is $L^{CLIP}(\theta) = \hat{E}t[\min(r_t(\theta)\hat{A}t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}t)]$

$r_t(\theta) = \frac{\pi_\theta(A_{AI,t}|S_t,\hat{G}t)}{\pi_{\theta\ old}(A_{AI,t}|S_t,\hat{G}t)}$ The probability ratio of the new and old policy ensures the smoothness of the policy update. $\hat{A}t$ is the advantage function (which measures the extent to which the action payoff is better than the average), is calculated using Generalized advantage Estimation (GAE). ϵ is the clipping coefficient that limits the magnitude of policy updates, preventing training oscillations caused by excessively large single updates (Schulman et al., 2016).

Finally, the policy and value network are jointly optimized to balance policy improvement, value fitting, and exploration.

3 Implementation

The above design shows the project needs to develop 3 parts including frontend, backend and model training, and all the developments are based on the tool of VScode.

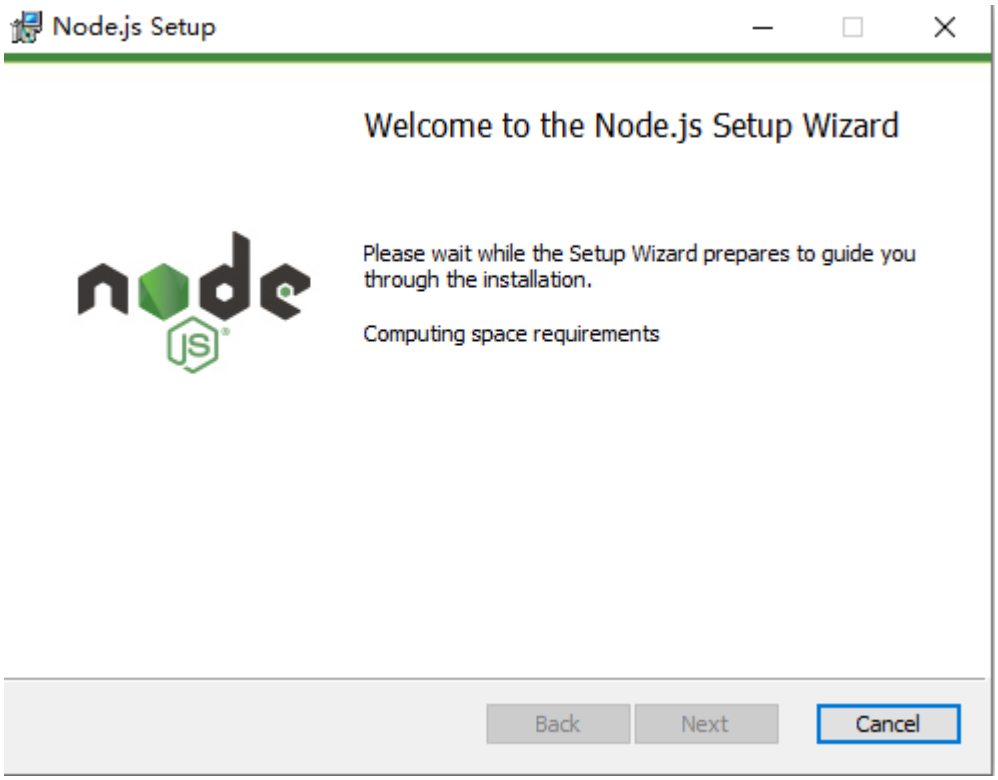
3.1 Front-end development

The front-end was built based on Vue2+Three.js, the MVVM architecture was used to decouple the interface and logic, and the development of common interactive components was completed

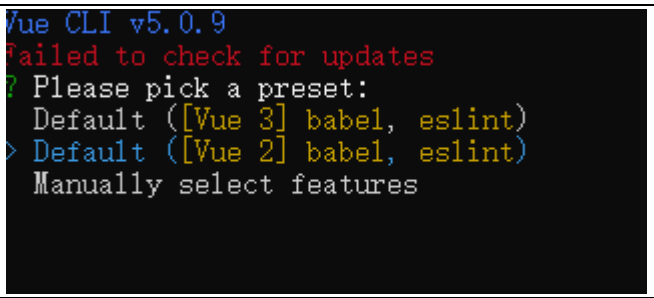
through Element UI. The 3D scene rendering, block model loading, mouse drag interaction and collision detection were realized based on Three.js. The ray casting mechanism was used to realize the block picking, and the grid alignment algorithm was used to ensure the standard position of the block. The RESTful interface is encapsulated based on Axios to realize core communication such as game state synchronization and AI decision request with the backend. At the same time, the local game state management is realized through sessionStorage.

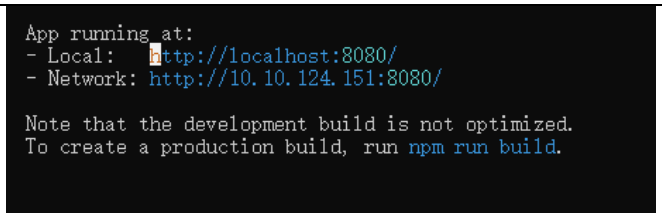
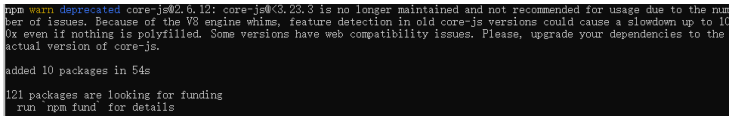
3.1.1 Environment set up

The node.js is the foundation to program based on vue2, which should be installed at first.



Then it is possible to set up a project based on the command of “vue create code”, and based on the “npm run serve” in cmd, the project would be started direct and visible in web browser. The it is very simple to import Element UI and three.js based on the “npm i” command.

vue create code (choose vue2 in this case)	

	
npm run serve	 
npm i element-ui three -- save	

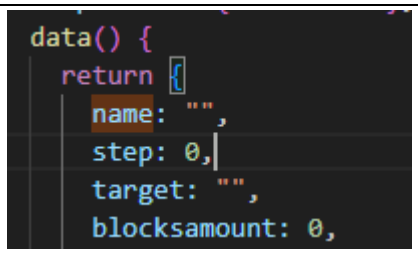
3.2.1 Page development

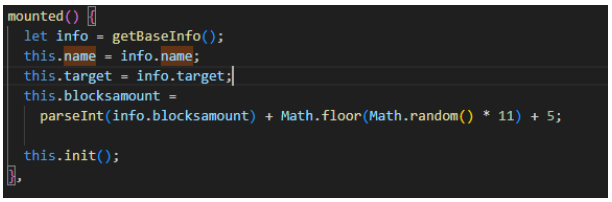
Based on the design, the whole project structure could be found in appendix 8.

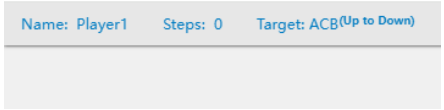
Taking the single game (home/index.vue) as an example, it shows how to develop and display 3D in a page.

Compared with common html page, the vue page uses the two-way binding variable, for example, when the data of name, target and blocks amount changes, the display changes directly.









In addition, except the common html item like span, it also consists of the special item like el-dropdown, which is the item from element UI, in this case, it refers to a dropdown list, these basic

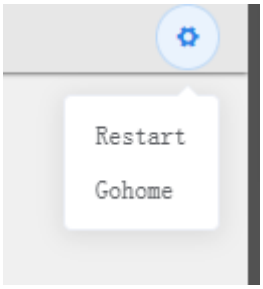
controls reduce page development time.

In Vue, it has different lifecycles, such as the mounted one in this example, which is the js equivalent of the onload method. All methods are enclosed in the “methods” block, and methods and variables are called by prepending the “this” scope.

<table>
<tr>
<td>

```
<el-dropdown trigger="click" @command="handleCommand">
  <el-button icon="el-icon-s-tools" circle></el-button>
  <el-dropdown-menu slot="dropdown">
    <el-dropdown-item command="restart">Restart</el-dropdown-item>
    <el-dropdown-item command="gohome">Gohome</el-dropdown-item>
  </el-dropdown-menu>
</el-dropdown>
```

```
handleCommand(command) {
  if (command === "restart") {
    // reload
    window.location.reload();
  } else if (command === "gohome") {
    // back to home
    this.$router.push("/main");
  }
},
```

</td>
<td> 
</td>
</tr>
</table>

In addition, an Axios-based interface module is encapsulated to handle all requests, then for each request, it is only necessary to pass in the url and parameter.

<table>
<tr>
<td>

```
export function createGameAPI(data) {
  return request({
    url: "/create-game",
    method: "post",
    data,
  });
}
```

```
// Call create game API
createGameAPI({
  name: playerName,
  blocksamount: blocksamount,
  mode: "race",
})
  .then((response) => {
    this.$message.success("Game created successfully!");
```

</td>
<td>It calls the api of “/create-game” and pass in the name, blocksamount and mode to start a game.</td>
</tr>
</table>

For multiplayer, the logic is that when a party starts the game as host, his interface goes into a waiting state, while his game is also visible in the hall, and when participants join, the game starts.

Take multiplayer racing mode for example:

<table>
<tr>
<td>

```
// Call create game API
createGameAPI({
  name: playerName,
  blocksamount: blocksamount,
  mode: "race",
})
  .then((response) => {
    this.$message.success("Game created successfully!");
    // Save the game ID and display the wait dialog
    this.currentGameId = response.data.game_id;

    this.waitingDialogVisible = true;
    this.loopStatus();
    // this.$router.push({ path: "/racevsplayer" });
    // this.$router.push({ path: "/game-room", query: { gameId: response.data.game_id } });
  })
  .catch((error) => {
    this.$message.error(
      "Failed to create game: " + (error.message || error)
    );
  });
},
```

</td>
<td>The host create a game and enter the waiting status</td>
</tr>
</table>

<table>
<tr>
<td>

```
<el-table
  :data="gameList"
  v-loading="loading"
  style="width: 100%"
  :empty-text="'No games available'"
>
  <el-table-column prop="creator" label="Name"> </el-table-column>

  <el-table-column prop="game_id" label="Game ID"> </el-table-column>
  <el-table-column prop="mode" label="Mode"> </el-table-column>

  <el-table-column prop="blocksamount" label="Blocks Amount">
  </el-table-column>
  <el-table-column prop="status" label="Status">
    <template slot-scope="scope">
      <el-tag
        :type="scope.row.status === 'waiting' ? 'success' : 'info'"
        size="small"
      >
        {{ scope.row.status }}
      </el-tag>
    </template>
  </el-table-column>
```

```
loadGameHall() {
  this.loading = true;

  getHallAPI().then((response) => {
    this.gameList = response.data || [];
    this.loading = false;
  });
},
```

</td>
<td>Based on the table from Element UI, the others could see the host in the game hall.</td>
</tr>
<tr>
<td>

```
<el-table-column label="Operate" width="120">
  <template slot-scope="scope">
    <el-button
      type="primary"
      size="mini"
      :disabled="scope.row.status !== 'waiting'"
      @click="joinGame(scope.row)"
    >
      Join
    </el-button>
  </template>
</el-table-column>
```

</td>
<td>Then by joining the game, both host and participate would be redirected to the game page, and in this case, it is the race mode page.</td>
</tr>
<tr>
<td>

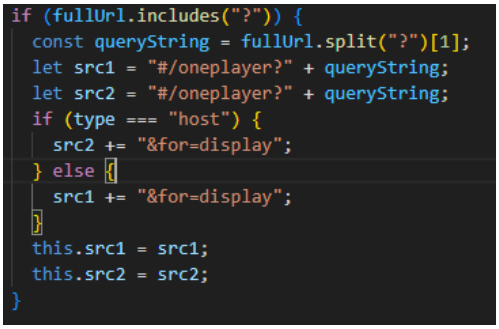
```
joinGame(game) {
  const playerName = localStorage.getItem("playerName");
  if (!playerName) {
    this.$message.error("Player name not found, please restart the game");
    return;
  }
  joinGameAPI({
    gameid: game.game_id,
    playerName: playerName,
    userid: this.userid,
  }).then((res) => {
    this.$message.success(`Successfully joined game ${game.game_id}`);
    console.log("Join game response:", res.data);

    // Jump to different pages depending on the game mode
    if (this.gameMode === "vsmode") {
      // VS mode
      this.$router.push({
        path: "/vsvsplayer",
        query: {
          gameId: res.data.game_id,
```

</td>
<td></td>
</tr>
<tr>
<td>

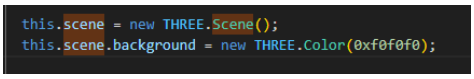
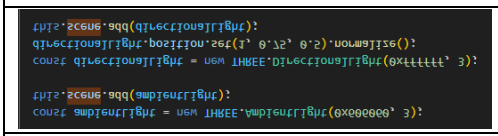
```
<div class="playground host">
  <iframe
    width="100%"
    height="100%"
    frameborder="0"
    scrolling="false"
    :src="src1"
  ></iframe>
</div>
<div class="playground join">
  <div class="playground-cover">
    <i class="el-icon-coffee-cup"></i> Dueler
  </div>
  <iframe
    width="100%"
    height="100%"
    frameborder="0"
    scrolling="false"
    :src="src2"
  ></iframe>
</div>
```

</td>
<td>The race mode would display the host and participates window in one page, and makes the different display based on the types.</td>
</tr>
</table>

	
 	The game is played in this way, every 1 second to get the progress of the opponent's moving block and determine whether the opponent is winning, and every time you move a block, you send the full block position to the background and determine whether you are winning.

3.2.2 3D development

For example, the 3D development based on three.js is completely different from the common html, which requires to have a scene at first, and then to add the light, renderer to display the objects (block in this case) in the scene.

	Create scene
	Add Light, and there are the different lights in three.js.
	Add blocks into the scene.

In addition, the block cannot form mouse event directly like click, so it uses a ray like flashlight to move with the mouse, and return the contacted items at the same time, and then proceed

functions based on the property of block like name.

Taking the function of moving a block as an example, it requires to bind 2 functions to the block including mousedown and mousemove (first click to pick up the block, second click to drop the block).

<table>
<tr>
<td>

```
onPointerDown(event) {

  if (event.button !== 0) return;

  this.pointer.set(
    (event.clientX / window.innerWidth) * 2 - 1,
    -(event.clientY / window.innerHeight) * 2 + 1
  );

  this.raycaster.setFromCamera(this.pointer, this.camera);
  const intersects = this.raycaster.intersectObjects(this.objects, false);
```

```
// The selection logic is handled only when a square is clicked
if (intersects.length > 0 && intersects[0].object !== this.plane) {
  const currentBlock = intersects[0].object;
  const blockPosition = currentBlock.position;
  const hasBlockAbove = this.checkBlockAbove(blockPosition);

  // Reset previous selection state
  this.resetSelectedBlock();

  // Remove any existing preview block
  if (this.previewBlock) {
    this.scene.remove(this.previewBlock);
    this.previewBlock = null;
  }

  if (hasBlockAbove) {
    // Use Vue message to notify
    if (this.$message) this.$message.error("This block cannot be moved");
  } else {
    // Set new selection state
    this.selectBlock(currentBlock);
  }
}
```

```
// If you already have the selected module and preview body, move it
if (this.selectedBlock && this.previewBlock) {
  // Move the selected block to the preview block position
  this.selectedBlock.position.copy(this.previewBlock.position);

  // Remove the preview block
  this.scene.remove(this.previewBlock);
  this.previewBlock = null;

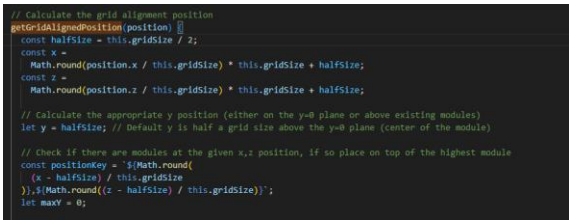
  // Reset selected state style
  this.resetSelectedBlock();
  this.step++;
  // Check if won
  this.showWinDialog();
  return;
}
```

</td>
<td>When the mouse is pressed, the ray is added, and if the ray touches the target block, the block is ready to be manipulated, including changing the color of the block and adding it to the selected list.
If the block has been added to selected list, click again would move the position, and release the selected list.</td>
</tr>
<tr>
<td>

```
// If there is a preview block, update its position
if (this.previewBlock) {
  if (intersects.length > 0) {
    const intersectPoint = intersects[0].point;
    const alignedPos = this.getGridAlignedPosition(intersectPoint);
    this.previewBlock.position.copy(alignedPos);
  }
} else if (intersects.length > 0) {
  const intersect = intersects[0];
  if (intersect.object !== this.plane) {
    const currentBlock = intersect.object;
    const blockPosition = currentBlock.position;
    const hasBlockAbove = this.checkBlockAbove(blockPosition);

    if (hasBlockAbove) {
      this.setBlockColor(currentBlock, 0xff0000);
    } else {
      this.setBlockColor(currentBlock, 0x00ff00);
    }
  }
}
```

</td>
<td>To prevent the block from moving randomly and losing its alignment, the getGridAlignedPosition method is added so that it can automatically align it to the grid.
The mouse movement will continuously position the mouse to the grid, so that the selected module can always follow the mouse position.</td>
</tr>
</table>



```
// Calculate the grid alignment position
getGridAlignedPosition(position) {
  const halfSize = this.gridSize / 2;
  const x =
    Math.round(position.x / this.gridSize) * this.gridSize + halfSize;
  const z =
    Math.round(position.z / this.gridSize) * this.gridSize + halfSize;

  // Calculate the appropriate y position (either on the y=0 plane or above existing modules)
  let y = halfSize; // Default y is half a grid size above the y=0 plane (center of the module)

  // Check if there are modules at the given x,z position, if so place on top of the highest module
  const positionKey = `${Math.round(
    (x - halfSize) / this.gridSize
  )},${Math.round((z - halfSize) / this.gridSize)}`;
  let maxY = 0;
```

3.2 Backend development

The backend system is implemented using Flask, a lightweight Python web framework, in main.py. The backend architecture is organized around two primary categories of API endpoints: game control interfaces and AI model-related interfaces. This decoupled design fully reflects the dual core attributes of the system, which not only completes the whole process management of multiplayer game sessions, but also realizes the deep integration of sophisticated AI opponents.

3.2.1 Overall Architecture

The backend maintains following structures to manage different game modes:

- game_hall: A list storing standard multiplayer games (race mode and versus mode)
- dueler_games: A dictionary storing AI-opponent games where players compete against AI
- vsai_games: A dictionary for games between AI

This system is compatible with multiple sets of block configurations, covering three different scene specifications of 5, 6, and 7 blocks. Each configuration needs to match the model that has been trained independently. The caching mechanism is designed for global model instances.

```
# Global model instances for different block configurations
_final_model = None       # 5-block interference model
_goal_predictor = None    # 5-block goal predictor
_final_model_6 = None     # 6-block interference model
_goal_predictor_6 = None # 6-block goal predictor
_final_model_7 = None     # 7-block interference model
_goal_predictor_7 = None # 7-block goal predictor
```

3.2.2 Game Control Interfaces

Take the /create-game api as an example, which completely presents the core business logic of game initialization. When a player initiates a new game creation request, the system must generate the target configuration parameters according to the game mode selected by the player.

```python
@app.route('/create-game', methods=['POST'])
def create_game():
    """New game creation interface"""
    data = request.form if request.form else request.json
    creator_name = data.get('name')
    blocksamount = data.get('blocksamount')
    mode = data.get('mode')

    all_letters = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    game_id = str(uuid.uuid4())[:8]

    if mode == 'vsmode':
        # VS mode: Each player gets a different target
        target_host = "".join(random.sample(
            all_letters[:int(blocksamount)],
            int(blocksamount)
        ))
        target_participant = "".join(random.sample(
            all_letters[:int(blocksamount)],
            int(blocksamount)
        ))
        # Ensure targets are different
        while target_participant == target_host:
            target_participant = "".join(random.sample(
                all_letters[:int(blocksamount)],
                int(blocksamount)
            ))
```

In the design of the battle mode, a key decision was to adopt a turn-based mechanism. There was an early attempt to support simultaneous action, but this caused race conditions that caused both players to manipulate blocks at the same time, creating an illegal game state. The turn-based mechanism solves this problem well. Maintain a single shared game state, and use turn validation logic to ensure that only players in the current turn can modify their position information.

```
new_game = {
    "game_id": game_id,
    "creator": creator_name,
    "blocksamount": blocksamount,
    "mode": mode,
    "target_host": target_host,
    "target_participant": target_participant,
    "status": "waiting",
    "participants": [],
    "current_turn": "host",  # Host moves first
    "block_positions": {
        "host": {"positions": [], "step": 0, "win": False},
        "participants": {"positions": [], "step": 0, "win":
    }
}
```

3.2.3 AI Model Integration

Taking 6 blocks mode in vs dueler scenario as an example, this module is the most technically complex component of the whole system, and its core logic is implemented in the /get-dueler-next-move interface. The interface schedules three independent AI core capabilities: target prediction, jamming strategy, and camouflage deception.

The system uses a lazy loading scheme, which only loads the necessary components for reasoning, effectively reducing the memory occupation of the system.

```
def load_models(blocksamount=5):
    """Load AI models (one-time initialization)"""
    global _final_model_6, _goal_predictor_6

    if blocksamount == 6:
        if _final_model_6 is None:
            checkpoint_path = os.path.abspath(
                "dueler/block6/interference_6blocks_checkpoints/checkpoint_000400"
            )
            try:
                learner_path = os.path.join(
                    checkpoint_path,
                    "learner_group",
                    "learner",
                    "rl_module"
                )

                if os.path.exists(learner_path):
                    from ray.rllib.core.rl_module.rl_module import RLModule
                    _final_model_6 = RLModule.from_checkpoint(learner_path)
                    print("[AI Interference Model] Loaded (6-block expert)")
```

The Decision execution process of AI strictly follows the priority hierarchy system of Goal Prediction →Interference Decision→Deception Mechanism.

Goal Prediction

The AI first predicts the player's goal based on the player's historical sequence of actions. Target prediction occasionally returned target results that did not match the number of squares required for the game. To solve this problem, the project first checks the legitimacy of the predicted output results, and then completes the calibration and adjustment of the predicted results by truncating the redundant content and filling the missing with the available blocks.

```
if current_goal_predictor and player_move_history and len(player_move_history) > 0:
    try:
        predictions = current_goal_predictor.predict(
            player_move_history,
            top_k=1
        )
        if predictions and len(predictions) > 0:
            predicted_target = predictions[0].get('goal', None)
            confidence = predictions[0].get('confidence', 0.0) / 100.0

            # Validate prediction matches block count
            if predicted_target and len(predicted_target) != blocksamount:
                print(f"[AI] Warning: Predicted target has {len(predicted_target)} blocks, expected {blocksamount}")
                # Adjust prediction...
```

Interference Decision

Based on the prediction target of the player, the AI constructs the corresponding observation vector for the interference model, and then the interference model outputs the action instructions in the range of [0,12].

- 0: No interference
- 1-6: Interfere with specific blocks A-F
- 7-12: Move AI's own blocks A-F

```
# Observation space for 6-block game: 64 dimensions
# 18 (player positions) + 18 (AI positions) +
# 12 (player goal) + 12 (AI goal) + 4 (progress metrics)
obs = []

# Player block positions (normalized)
for i in range(num_blocks):
    if i < len(current_positions):
        pos = current_positions[i]
        obs.extend([
            pos['x'] / 20.0,
            pos['y'] / 20.0,
            pos['z'] / 10.0
        ])
```

Deception Mechanism

When the model decides not to perform the jamming action, the AI will have a probability to perform the spoofing action. The probability of spoofing will decrease as the AI stacks up. When the AI is close to completing its own stacked goal, it focuses on completing its own goal; In the early stages of the game, the AI actively misleads the player

```
should_deceive = False
if player_critical:
    # Player about to win - no deception
    should_deceive = False
elif ai_progress_ratio < 0.4:
    # Early game: 60% chance to deceive
    should_deceive = random.random() < 0.6
elif ai_progress_ratio < 0.7:
    # Mid game: 30% chance
    should_deceive = random.random() < 0.3
```

Collision Detection and Resolution

Finally, the avoidance processing of overlapping squares. At the beginning of the project, we tried to avoid the collision problem by pre-checking before moving. However, due to the timing problem in multi-player game scenarios, the protection effect of this scheme cannot meet the requirements of system stability. The final scheme adopted by the current system is to verify the legitimacy of the position information of all the blocks after each movement operation is completed.

```python
def validate_and_fix_positions(positions):
    """Validate and fix block positions to prevent overlap"""
    position_map = {}

    for p in positions:
        pos_key = (int(p['x']), int(p['y']), int(p['z']))
        if pos_key not in position_map:
            position_map[pos_key] = []
        position_map[pos_key].append(p['block'])

    # Find conflicts
    conflicts = {pos: blocks for pos, blocks in position_map.items()
                 if len(blocks) > 1}

    if not conflicts:
        return positions

    # Resolve by moving conflicting blocks
    for conflict_pos, conflict_blocks in conflicts.items():
        blocks_to_move = conflict_blocks[1:]  # Keep first, move others
        for block_name in blocks_to_move:
            # Find empty position using spiral search...
```

In summary, the back-end system of this project successfully realized the deep integration of reinforcement learning models and real-time multi-player game system. Key results of the project include an adaptive model loading mechanism to adapt to different block configurations, a highly robust collision handling scheme, and a three-tier AI decision architecture. The challenges faced in the development process mainly focused on the data synchronization problem in multi-player game scenarios and the consistency verification of model output results.

3.3 Development of AI

The AI system designed in this study contains two basic capabilities, namely goal prediction and interference strategy. The core function of goal prediction is to infer the goal configuration of the game based on the historical operation sequence of the player. interference strategy is responsible for deciding when to interfere with the progress of the game and how to implement it. The implementation of the two competencies depends on the training data that are specially constructed to simulate real game scenarios. Here again, the 6 blocks mode vs dueler scenario is used as an example.

3.3.1 Training Data Generation

The training of dual AI models (Goal Prediction Model and Interference Strategy Model) relies on two different datasets: Non-Interference Dataset (used for supervised learning of target prediction to capture player stacking behavior in the absence of external Interference) and Interference Dataset (used for reinforcement learning of adversarial strategies to simulate the human-machine confrontation process in a shared space).

Both datasets were generated by Python scripts (makePlayHistory_no_interference.py and makePlayHistory_interference.py), which followed the physical rules to verify the game logic constraints, and each of them makes 10000 data.

3.3.1.1 Non-Interference Data Generation (for Goal Prediction Model)

The core task of the object prediction model isto infer the expected final configuration of the player by observing part of the game sequence, which is implemented in makePlayHistory_no_interference.py. The construction strategy of the non-interference dataset, prioritizing diversity (30 permutations ×10 square configurations × deception action patterns) over data scale, enables the target prediction model to learn from it and identify the true intention of the player. Considering the balance between feasibility and coverage of computing resources, this study did not perform the study on all 720 possible permutations of ABCDEF (6! = 720), instead screening 30 representative configurations.

```
TARGET_PERMUTATIONS = [
    "ABCDEF",  # Fully sequential (baseline)
    "FEDCBA",  # Fully reversed (maximum difficulty)
    "ABCDFE",  # Last two swapped (common mistake pattern)
    "ABCEDF",  # E and D swapped (adjacent swap)
    "ABCFED",  # Last three reversed (partial reversal)
    "ABDCEF",  # C and D swapped
    "ACBDEF",  # B and C swapped
    # ... 23 more strategically selected permutations
    "AEFBCD"   # E and F in middle positions
]
```

The 30 configurations covers enough categories:

- Sequential patterns (ABCDEF, FABCDE): Low difficulty baselines
- Reversal patterns (FEDCBA, ABCFED): High difficulty extremes
- Adjacent swaps (ABCEDF, ABDCEF): Common intermediate states

- Complex permutations (BDEFCA, DBFACE): Diverse training examples

This distribution ensures the model encounters varying difficulty levels rather than overfitting to alphabetical or reversed sequences.

The single training sample designed for the data set in this study can fully reproduce the whole game process from game initialization to goal achievement. In the process of sample generation, several core mechanisms are integrated, including Variable Block Count with Distractors, Initial Position Randomization. And Stepwise Construction with Deceptive Moves.

Variable Block Count with Distractors

This mechanism introduces distractor blocks (G-Z). The total number of blocks in a single game can be set to 6-15, corresponding to 0-9 distractor blocks, to truly simulate the real game scene where players need to circumvent obstacles to complete the stacking operation.

```python
def generate_random_block_list() -> List[str]:
    """Generate random block list (6-15 blocks, must include ABCDEF)"""
    num_blocks = random.randint(MIN_BLOCKS, MAX_BLOCKS)  # 6-15
    # Must contain ABCDEF
    block_list = ["A", "B", "C", "D", "E", "F"]
    # Add extra blocks (G-Z) until reaching specified count
    if num_blocks > 6:
        extra_blocks = [c for c in ALL_BLOCK_CHARS if c not in block_list]
        random.shuffle(extra_blocks)
        block_list += extra_blocks[:num_blocks - 6]
    return block_list
```

Initial Position Randomization

All blocks start at the ground level (z=0) and have non-overlapping x/y coordinates. The current generation scheme can ensure that all blocks are accessible in the initial state.

```python
def generate_initial_positions(block_list: List[str]) -> List[Dict]:
    """Generate initial positions for all blocks: z=0, non-overlapping x/y"""
    positions = []
    used_coords = set()
    for block in block_list:
        while True:
            x = random.randint(1, BOARD_SIZE)  # 20x20 board
            y = random.randint(1, BOARD_SIZE)
            if (x, y) not in used_coords:
                used_coords.add((x, y))
                break
        positions.append({"block": block, "x": x, "y": y, "z": 0})
    return positions
```

Stepwise Construction with Deceptive Moves

The innovation of this study is to generate move sequences that fit the real game scenario, and the non-optimal player behaviors such as false starts and distractor actions are fully integrated in the sequences. Instead of only generating shortest path solutions that conform to the optimal logic.

```
def generate_steps_to_target(initial_blocks: List[Dict],
                             target_perm: str) -> List[List[Dict]]:
    """Generate step sequence with interference and fake moves"""
    steps = [initial_blocks.copy()]
    current_blocks = initial_blocks.copy()

    # Phase 1: Initial distractions (move non-target blocks)
    num_initial_distractions = random.randint(1, 3)
    for _ in range(num_initial_distractions):
        result = safe_move_non_target()  # Move G-Z blocks randomly
        if result:
            current_blocks = result
            steps.append(current_blocks.copy())

    # Phase 2: Move bottom block, possibly to wrong position first
    bottom_block = target_perm[5]  # Bottom of stack (z=0)
    if random.random() < 0.3 and is_block_movable(bottom_block, current_blocks):
        # 30% chance to first move to incorrect position (fake move)
        fake_x = random.randint(1, BOARD_SIZE)
        fake_y = random.randint(1, BOARD_SIZE)
        if (fake_x != target_base_x or fake_y != target_base_y):
            current_blocks = move_block(current_blocks, bottom_block,
                                        fake_x, fake_y, 0)
            steps.append(current_blocks.copy())

    # Then move to correct position
    if is_block_movable(bottom_block, current_blocks):
        current_blocks = move_block(current_blocks, bottom_block,
                                    target_base_x, target_base_y, 0)
        steps.append(current_blocks.copy())
```

The generation is as followed:

- Initial distractions (1-3 moves): Random movement of non-target blocks
- Bottom layer placement (target_perm[5]): May include fake positioning (30% probability)
- Mid-construction interference: Random non-target movements
- Second layer (target_perm[4]): Possible fake moves (20% probability)
- Third layer (target_perm[3]): Possible fake moves (20% probability)
- Additional interference (0-2 moves): More distractor movements
- Fourth layer (target_perm[2]): Direct placement
- Fifth layer (target_perm[1]): Direct placement

- Top layer (target_perm[0]): Final placement

The fake move mechanism creates training samples where early move sequences are ambiguous. Taking the game scenario where the target stacking sequence is "FEDCBA" as an example, the player may first move the F block to the coordinate (5,5) in the actual operation, and then relocate it to the final stacking position (10,10). Such non-optimal path behavior sequences are completely included in the coverage of the sample generation.

In addition, all generated square locations must satisfy two core physical constraints, the overlap constraint and the floating constraint. The current optimal scheme will perform full legitimacy verification after the completion of each move generation. Once it is detected that it does not conform to the constraints, it will be eliminated as a whole and not included in the final data set..

```python
def is_position_valid(block_pos: Dict, all_blocks: List[Dict]) -> bool:
    """Check single block position validity: no overlap, no floating"""
    # Check overlap (same x/y/z)
    for b in all_blocks:
        if b["block"] == block_pos["block"]:
            continue
        if (b["x"] == block_pos["x"] and b["y"] == block_pos["y"] and
            b["z"] == block_pos["z"]):
            return False

    # Check floating (z>0 requires support directly below)
    if block_pos["z"] > 0:
        has_support = False
        for b in all_blocks:
            if (b["x"] == block_pos["x"] and b["y"] == block_pos["y"] and
                b["z"] == block_pos["z"] - 1):
                has_support = True
                break
        if not has_support:
            return False
    return True
```

Summary results of the generated scripts are shown in appendix 2. The distribution of the number of squares and the target arrangement is nearly uniform (variance is ±5% and ±0.5%, respectively), which effectively ensures that the training process is unbiased.

Problem Description	**Failed Attempts**	**Final Solution**
Premature target achievement: Some step sequences meet the target in advance	Simply deleting the abnormal steps (causes step sequence	Add assertion checks to filter invalid data; regenerate data if the assertion fails

	discontinuity)	
Unbalanced distribution: Some permutations are selected too frequently	Manually set weights (hard to tune and inflexible)	Use 30 random permutations; increase sample size to 100,000 to dilute randomness
Floating/overlapping blocks: Invalid block positions due to random coordinate generation	Reducing randomness of coordinates (reduces data diversity)	Use used_coords to avoid x/y overlap; is_position_valid checks floating blocks
Excessive fake moves: Model training fails to capture valid stacking behavior	Adding unlimited fake moves for all core blocks (simulates overly irrational human)	Control fake move probability (30% for bottom blocks, 20% for middle blocks) and limit fake move count

3.3.1.2 Interference Data Generation (for Interference Strategy Model)

The interference data is generated by the makePlayHistory_interference.py script, which is designed for the adversarial strategy training of PPO reinforcement learning model. The core game scenario is: the player completes stacking 4-5 core blocks (critical progress has been made) → The AI does interference by removing the top block of the player → The AI completes its goal stacking The player loses the game and the AI wins.

The key parameters controlling the dynamics of this game are as follows:

```
# Interference trigger thresholds
INTERFERENCE_MIN_PROGRESS = 4  # AI waits until player stacks ≥4 blocks
INTERFERENCE_MAX_PROGRESS = 5  # AI must interfere before player stacks 6 blocks
INTERFERENCE_WINDOW = 20       # AI has 20 moves post-interference to win
MIN_STEPS_BEFORE_INTERFERENCE = 5   # Minimum player moves before interference
MAX_STEPS_BEFORE_INTERFERENCE = 15  # Maximum player moves before interference
```

A threshold of 4-5 blocks represents critical nodes where the player has invested a lot of effort (around 67%-83% completion) but has not yet won. Jamming too early (1-2 blocks) does not provide enough training signal, and jamming too late (6 blocks) prevents the AI from winning. This threshold interval maximizes the strategic value of jamming timing

```python
from itertools import permutations

TARGET_BLOCKS = ["A", "B", "C", "D", "E", "F"]
TARGET_PERMUTATIONS = ["".join(p) for p in permutations(TARGET_BLOCKS)]  # All 720

# Ensure player and AI have different competing objectives
player_target = select_target_permutation()
ai_target = select_target_permutation()
while ai_target == player_target:
    ai_target = select_target_permutation()
```

The initial implementation allowed the player and the AI to share the same goal, which created a meaningless game where interference always paid off. By introducing a while loop to ensure that the goals of both sides are differentiated, this degradation problem is eliminated, forcing the AI to balance between disrupting the opponent and advancing its own progress. The player builds phase uses a proven build strategy, and the logic of steps_per_block distributes moves across blocks rather than stacking blocks in a single move. For example, with target_progress=4 and num_steps=12, the player takes approximately 3 moves per block (12 ÷ 4 = 3), which creates a gradual construction like a real human.

```python
def generate_player_progress_steps(
    initial_blocks: List[Dict],
    player_target: str,
    target_progress: int  # 4 or 5
) -> Tuple[List[List[Dict]], int]:
    """Generate player moves toward target until reaching specified progress"""
    steps = [initial_blocks.copy()]
    current_blocks = initial_blocks.copy()

    # Random stacking base position
    target_base_x = random.randint(1, BOARD_SIZE)
    target_base_y = random.randint(1, BOARD_SIZE)

    # Build from bottom up: player_target="ABCDEF" → F(z=0), E(z=1), ..., A(z=5)
    blocks_to_stack = []
    for i in range(target_progress):
        blocks_to_stack.append((player_target[5 - i], i))  # (block_id, target_z)

    # Generate realistic move count (5-15 moves)
    num_steps = random.randint(MIN_STEPS_BEFORE_INTERFERENCE, MAX_STEPS_BEFORE_INTERFERENCE)
    steps_per_block = max(1, num_steps // target_progress)
```

After generation, the validation strictly limits the range of learning samples of the model, and only keeps the effective samples that trigger interference behavior at the preset key nodes for the model to learn.

```python
# Verify player achieved required progress (4-5 blocks)
if actual_progress < INTERFERENCE_MIN_PROGRESS or actual_progress > INTERFERENCE_MAX_PROGRESS:
    continue  # Discard and regenerate
```

The AI's interference follows the 2 approaches:

Stage 1 - Disruption: Identify and dismantle the player's stack:

```
def generate_ai_interference_steps(
    blocks_after_player: List[Dict],
    player_target: str,
    ai_target: str
) -> List[List[Dict]]:
    """Generate AI interference and self-completion steps"""
    steps = []
    current_blocks = blocks_after_player.copy()

    # Stage 1: Interfere with player
    player_progress = get_progress_towards_target(current_blocks, player_target)
    if player_progress >= 3:
        # Target second-highest block in player's stack
        top_block_id = player_target[5 - player_progress + 1]
        if is_block_movable(top_block_id, current_blocks):
            # Move to random position, breaking player's progress
            new_x = random.randint(1, BOARD_SIZE)
            new_y = random.randint(1, BOARD_SIZE)
            current_blocks = move_block(current_blocks, top_block_id, new_x, new_y, 0)
            steps.append(current_blocks.copy())
```

Instead of randomly choosing blocks to interfere with, the algorithm precisely targets the next highest block in the player's stack, the one at player_target[5 - player_progress + 1]. This target selection strategy can achieve the maximum interference effect for the following reasons:

- The top block (z=5) is typically already accessible for the player to move
- Removing the second-highest block (z=4 when progress=5) causes the top block to become unsupported, potentially invalidating it
- It forces player to reform the 2+ layers

Stage 2 - Self-goal achievement

The AI builds its own stack from scratch at a different (x, y) location, ensuring no spatial conflict with the disrupted player stack.

```
# Stage 2: AI completes its own goal
ai_base_x = random.randint(1, BOARD_SIZE)
ai_base_y = random.randint(1, BOARD_SIZE)

for i in range(6):
    block_id = ai_target[5 - i]  # Bottom to top
    target_z = i

    if is_block_movable(block_id, current_blocks):
        current_blocks = move_block(current_blocks, block_id, ai_base_x, ai_base_y, target_z)
        steps.append(current_blocks.copy())

        if is_target_achieved(current_blocks, ai_target):
            return steps  # AI wins
```

All generated samples are rigorously verified before being included in the final dataset.

```
# Verify final state meets adversarial requirements
final_state = ai_steps[-1] if ai_steps else player_steps[-1]
player_completed = is_target_achieved(final_state, player_target)
ai_completed = is_target_achieved(final_state, ai_target)

if player_completed or not ai_completed:
    continue  # Discard: must have player loss AND AI victory
```

This generation strategy finally builds a pure adversarial dataset: 100% of the samples in the dataset are scenarios in which the AI wins and the player loses.

The dataset is as followed:

```
=== Data Statistics ===
Total valid records: 100,000

Player progress before interference:
  Stacked 4 blocks: 82,816 samples (82.82%)
  Stacked 5 blocks: 17,184 samples (17.18%)

Move count statistics:
  Average total moves: 15.13
  Average player moves: 8.13
  Average AI moves: 7.00
  Minimum total moves: 10
  Maximum total moves: 21

Final outcome validation:
  Player completed task: 0 samples (expected 0) ✓
  AI completed task: 100,000 samples (expected 100,000) ✓

Target permutation diversity:
  Player target variety: 720/720 (100.0%)
  AI target variety: 720/720 (100.0%)
```

In this study, it was found that the data set showed a highly skewed distribution characteristics towards the 4-block interference scene, which was beyond the previous expectation. In the initial stage, the system is configured to generate an equal number of 4-block and 5-block interference scenes (the ratio of them is 50:50), but the validation rules of the dataset have a natural screening effect on the generated results. Specifically, the 5-block interference scenario has a significantly higher failure rate because it only takes one extra move to complete the target stack, making it

difficult for the AI to completely block the player's progress. The final dataset shows an unbalanced distribution of 82.82% samples with 4-square scenes and 17.18% samples with 5-square scenes, which has a positive effect on model training. The model can be exposed to more effective samples of optimal intervention timing, rather than high-risk late intervention cases.

The statistical results of the dataset show that the average total number of moves in a single game is about 15.13 (including 8.13 moves for the player and 7.00 moves for the AI), which is shorter than the 18 steps estimated in the previous period. The compression of game duration is since the checksum filtering mechanism preferentially retains the game samples that the AI efficiently completes the target stack. The distribution of the total number of moves in a game ranges from 10 to 21, which indicates that the game duration is within the reasonable operating range of a human player.

AI wins 100% of the games, which validates that the dataset only contains effective adversarial scenarios where the jamming strategy works successfully. In addition, the dataset achieves 100% coverage of all 720 possible permutations of player and AI targets, which ensures that the model is exposed to all possible target configurations and effectively avoids overfitting to limited scenarios.

3.3.2 Training Model

The AI system requires two distinct neural network models including a goal prediction model that infers player intentions from move sequences, and an interference strategy model that determines when and how to disrupt player progress. Both employ deep learning architectures but address different tasks. In this case it also takes the 6 blocks mode vs dueler as an example.

3.3.2.1 Goal Prediction Model Training

The AI system constructed in this study includes two sets of neural network models. One is a goal prediction model, which can infer the potential action intention of the player based on the game movement sequence. The other is interference strategy model, which is used to decide when to interfere with the progress of the game and how to implement it. Both sets of models are designed by deep learning architecture, and 6 blocks mode vs dueler mode is still used as an example.

```python
class GoalPredictionModel(nn.Module):
    def __init__(self, max_blocks=15, hidden_dim=256, num_heads=8,
                 num_layers=4, num_classes=30):
        super().__init__()

        # 1. Position encoder: (x,y,z) -> hidden_dim
        self.position_encoder = nn.Sequential(
            nn.Linear(3, 128),
            nn.ReLU(),
            nn.Linear(128, hidden_dim)
        )

        # 2. Block aggregator: combine all blocks per timestep
        self.block_aggregator = nn.Sequential(
            nn.Linear(hidden_dim * max_blocks, hidden_dim),
            nn.ReLU(),
            nn.Dropout(0.1)
        )

        # 3. Transformer encoder: process temporal information
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=hidden_dim,
            nhead=num_heads,
            dim_feedforward=hidden_dim * 4,
            dropout=0.1,
            batch_first=True
        )
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)

        # 4. Classification head
        self.classifier = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(0.1),
            nn.Linear(hidden_dim, num_classes)
        )
```

Transformer architectures excel at capturing long-distance dependencies in sequential data. Unlike recurrent neural networks (RNNs/LSTMs), Transformers process data at all time steps simultaneously through a self-attention mechanism, enabling the model to directly compare early and late movement operations without the information bottleneck problem. This property is critical for target prediction tasks because indicative moves can occur at any stage of the game - for example, placing square F at the bottom early or placing square A at the top late, both of which strongly suggest target alignment.

```python
def __getitem__(self, idx):
    game = self.data[idx]
    target_perm = game['target_permutation']
    target_label = self.perm_to_idx[target_perm]

    # 🔴 Critical fix: exclude final step (goal state)
    steps = game['steps'][:-1]

    # Ensure at least one step remains
    if len(steps) == 0:
        steps = game['steps'][:1]  # Keep at least initial state
```

The training flow adopts the standard supervised learning scheme, uses the cross-entropy loss as the loss function, and divides the test set and the training set by the ratio of 1:9.

```python
# Model: 2.1M parameters
model = GoalPredictionModel(
    max_blocks=15,      # Support up to 9 distractor blocks
    hidden_dim=256,     # Feature dimension
    num_heads=8,        # Multi-head attention
    num_layers=4,       # Transformer depth
    num_classes=30      # 30 curated permutations
)

# Optimization
criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=1e-4, weight_decay=0.01)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs)
```

```python
# Split the training and validation sets (90/10)
train_size = int(0.9 * len(dataset))
val_size = len(dataset) - train_size
```

After training, it is found that after only one epoch of training, the verification accuracy of the model reaches 96.50%, while the training accuracy is only 57.09%.

By the second epoch, the verification accuracy rises to 99.13%, which basically completes the classification task.

This performance may be due to the fact that even though the final target state has been removed from the training sequence (steps [:-1]), the penultimate step tends to contain 5 correctly stacked squares, leaving only 1 square to be placed. For most permutations, this five-square configuration is almost determinable-there's only one logical way to do it.

This high accuracy still has a positive effect on the target application scenario.

```
Epoch 1/50:
  Train - Loss: 1.5099, Acc: 57.09%
  Val   - Loss: 0.0909, Acc: 96.50%
  ✅ New best validation accuracy: 96.50%

Epoch 2/50:
  Train - Loss: 0.0571, Acc: 98.10%
  Val   - Loss: 0.0261, Acc: 99.13%
  ✅ New best validation accuracy: 99.13%
```

3.3.2.2 Interference Strategy Model Training

Second, unlike supervised object prediction tasks, jamming strategies require learning a set of sequential decision policies: at each time step, the AI must decide whether to disturb the player or continue to build its own stacked structure. This problem can be naturally modeled as a reinforcement learning (RL) problem and implemented by Proximal Policy Optimization (PPO) (train_interference.py) algorithm in RLlib framework.

```python
state_dim = 18 + 18 + 12 + 12 + 1 + 1 + 1 + 1  # = 64

self.observation_space = spaces.Box(
    low=-1, high=1,
    shape=(state_dim,),
    dtype=np.float32
)
```

```python
def _get_observation(self) -> np.ndarray:
    """Constructing observation vectors"""
    obs = []

    # 1. Player position (normalized)
    for pos in self.player_positions:
        obs.extend([
            pos['x'] / self.grid_size,
            pos['y'] / self.grid_size,
            pos['z'] / 10.0
        ])

    # 2. AI position (normalized)
    for pos in self.ai_positions:
        obs.extend([
            pos['x'] / self.grid_size,
            pos['y'] / self.grid_size,
            pos['z'] / 10.0
        ])

    # 3. Player target (simplified 12-dim embedding)
    player_target_hash = self._target_to_embedding(self.player_target)
    obs.extend(player_target_hash)
```

```python
# 4. AI target (simplified 12-dim embedding)
ai_target_hash = self._target_to_embedding(self.ai_target)
obs.extend(ai_target_hash)

# 5. Player current progress
player_progress = self._get_player_progress(self.player_positions, self.player_target)
obs.append(player_progress / 6.0)

# 6. Interference count (normalized)
obs.append(min(self.interference_count / 10.0, 1.0))

# 7. Current step ratio
obs.append(self.total_steps / self.max_steps)

# 8. Last action (normalized)
obs.append(self.last_action / 10.0)

return np.array(obs, dtype=np.float32)
```

```python
# Action space
# 0: Focus on own target (no interference)
# 1-6: Interfere with player's blocks A/B/C/D/E/F
# 7-12: Move own blocks A/B/C/D/E/F to target positions
self.action_space = spaces.Discrete(13)
```

Observation (64 dimensions for 6-block):

- Player block positions: 6 blocks × 3 coords = 18 dims
- AI block positions: 6 blocks × 3 coords = 18 dims
- Player goal embedding: 12 dims (one-hot + permutation encoding)
- AI goal embedding: 12 dims
- Progress metrics: 4 dims (player progress, interference count, step ratio, last action)

Action (13 discrete actions):

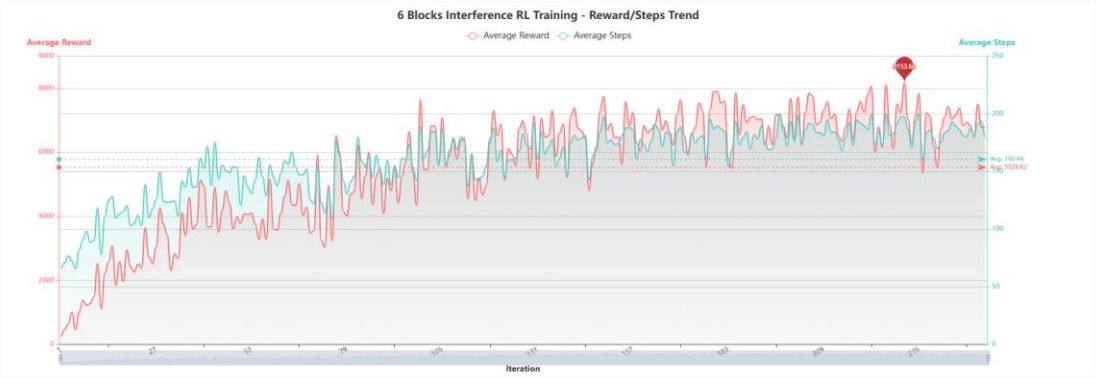
- Action 0: No interference (focus on self-construction)
- Actions 1-6: Interfere with player blocks A-F
- Actions 7-12: Move AI's own blocks A-F

The reward structure creates a multi-objective optimization problem (Appendix 3).

```
# Reward weight
self.reward_prevent_player = config.get("reward_prevent_player", 100.0)  # Successfully stopping the player
self.reward_ai_complete = config.get("reward_ai_complete", 80.0)  # AI completing its goal
self.reward_player_complete = config.get("reward_player_complete", -150.0)  # Player completing (failure)
self.reward_interference_timing = config.get("reward_interference_timing", 50.0)  # Correct timing interference (increased to 50)
self.reward_wrong_timing = config.get("reward_wrong_timing", -30.0)  # Wrong timing interference (increased penalty to -30)
self.reward_effective_interference = config.get("reward_effective_interference", 15.0)  # Effective interference (reduced to 15)
self.reward_progress_ai = config.get("reward_progress_ai", 5.0)  # AI progress reward
self.reward_step = config.get("reward_step", -0.1)
```

The interference timing bonus (+50) is calibrated to be significant but not overwhelming—pure interference without self-completion still yields negative cumulative reward due to step penalties and the need to balance both objectives.

PPO training ran for 1000 iterations (the iterations stop at 253 iterations because by testing the model on time, it finds the AI's rate for successfully prevented player reaches 94%, which is enough to play the game; continuing to train to enhance the AI's defense will cause the player's success rate to drop even further):



The policy demonstrates surprisingly strong initial performance with mean reward of 245.81, indicating effective reward shaping from the environment design. The reward improves dramatically by 1558% in the first 50 iterations (245.81 → 3798.31), while episode length increases from 65.9 to 120.5 steps—demonstrating the discovery of more complex but effective strategies that balance interference with self-completion.

4 Testing & Evaluation

Based on the design and object of this project, the test is divided into the usability testing based

on storyboard, the model usability testing and feedback from 10 users. In addition, due to the limitation of hardware (lack high-performance graphics cards), the test of response speed does not take into consideration.

4.1 Usability testing

According to the following usability tests, it is found that the project has high usability which could complete the different play modes.

The full-function usability test is completed based on the storyboard, covering four core modes: single-player training, multi-player online racing, human-computer racing, and human-computer confrontation. All core functions (game creation/joining, square dragging interaction, winner/opponent judgment, AI decision calling, online state synchronization) have passed the test, and details could be found in appendix 4.

4.2 Model usability testing

The accuracy of the model was tested, including the accuracy of the predicted target and the interference decision.

4.2.1 Prediction Accuracy

To comprehensively evaluate the usability and reliability of the target prediction model, this study conducts a systematic accuracy evaluation from three dimensions: memory ability, same-target generalization ability, and new target generalization ability.

This evaluation is based on the dedicated test suite test_goal_prediction_block6.py, which comprehensively tests the performance of the model for different scenarios. The specific design is as follows:

- Memorability: Three samples were randomly selected from each of the 30 trained target permutations to test the model's ability to reproduce the training data.
- Same-target generalization ability: The ability of the model to recognize the same target is evaluated in the presence of perturbations in the initial state by applying a random position offset of ±2 units to the opposite block location.

- New object generalization: Evaluate how well the model responds to unknown objects using five object permutations the model has never seen before.

In all tests, the Top-k accuracy index (K=1,3,5) is used to complete the prediction effect evaluation, and the core measure is whether the correct target appears in the Top K prediction results of the model output.

The test results in Appendix 5 show that for the core application scenario of this study (predicting objects from a known set of 30 permutations), the model shows excellent usability, with an overall accuracy of 98.9%. When the real target is present in the training set, the AI can reliably interpret and apply the predictions. However, the system has not yet designed a mechanism to identify and prompt new unknown targets. When it encounters targets outside the training set, it may output high confidence wrong prediction results, which will mislead the subsequent decision of the AI.

If there is more time, it would improve:

- A confidence threshold mechanism was added to mark the risk of the prediction results when the input pattern deviated significantly from the training data.
- Generate more examples of permutations to cover a wider target space (e.g., all 720 possible combinations of 6-square permutations).
- The model architecture was redesigned, and the fixed category classification task was replaced with the block-level ranking output to realize the real combinatorial generalization ability.

4.2.2 Accuracy of interference decision

The core design goal of the interference expert model is to autonomously advance and complete their own established task goals while preventing players from completing the game goals. In this evaluation, a special test script test_interference.py is used to load the best trained PPO model, checkpoint (iteration 231, this model is the best performance version, and the reward value is 8153.68). The full-dimensional performance test is completed in 100 randomly generated game scenarios.

Each episode consists of:

- A player tries to complete the goal of 6 blocks (A-F)
- An AI agent with its own conflicting target permutation

- A maximum of 200-time steps to prevent indefinite episodes
- The AI choosing between 13 possible actions: no interference (1), interfere with specific blocks (6), or move its own blocks (6)

The test measured four performance indicators:

- Prevention Success Rate: Percentage where the player failed to complete
- AI Completion Rate: Percentage of episodes where the AI achieved its own goal
- Interference Timing Accuracy: Whether interferences occurred at optimal moments
- Resource Efficiency: Average and interference count in one episode

The test results show that the model achieves an excellent success rate of 94% in preventing the player from completing the goal.

Outcome Category	**Episodes**	**Percentage**
AI Goal Completion	16/100	16.0%
Prevention by Timeout	78/100	78.0%
Player Completed	6/100	6.0%

This result proves that the model can effectively interrupt the player's progress rhythm through the combination of strategic interference and phased autonomous completion of goals. 78% of these blocks were due to game timeouts, indicating that the model has learned to impose a constant suppression on players, keeping their stacking progress below the completion threshold (only 4.10/6 blocks completed on average).

In addition, the model shows a strong ability to control the strategic timing, and the interference timing accuracy reaches 83.8% :

Timing Category	**Count**	**Percentage**
Player progress = 4	8,087/15,074	53.6%
Player progress = 5	4,542/15,074	30.1%
Other progress levels	2,445/15,074	16.2%
Correct Timing Rate	**12,629/15,074**	**83.8%**

This result indicates the model has successfully learned the optimal interference window, that 83.8% of interferences happens at critical moments (progress 4-5).

The limitations consist of 16% self-completion rate suggests the model prioritizes prevention over

self-achievement; 150.7 average interferences (81.7% of actions) indicate heavy reliance on defensive tactics; and 78% prevention via timeout suggests the model could benefit from more aggressive self-completion.

If given more time, I would implement the state-based policy selection, for example:

- If player progress < 3: Focus on self-completion (80% self-moves)
- If player progress 3-4: Balanced approach (50/50 split)
- If player progress ⩾ 5: Full prevention mode (90% interference)

4.3 User feedback

A total of 10 participants were recruited for the user testing, all participants signed a participant consent form (included in the Appendix 6) prior to testing, which clarified the testing purpose, activity process, data usage rules (anonymous collection and only for project research), and voluntary withdrawal rights.

The structured questionnaire included 8 Likert scale questions and 2 open-ended questions, with the core research questions targeting the experience of playing against Dueler in racing and confrontation modes (Question 6 and Question 7) in appendix 7.

User feedback is divided into positive subjective feelings and negative problems, all feedback is anonymized (marked by participant number P1-P10).

4.3.1 Positive Feedback

In the evaluation of the confrontation mode, 9 out of 10 subjects rated this mode as the most fun to play. Subjects generally said that the dynamic camouflage and interference behaviors exhibited by the AI brought a strong strategic game experience to the game. For example, subjects P2, P5, and P8 mentioned, "It is very unexpected and challenging for the AI to pretend to move irrelevant blocks at the beginning of the game, but suddenly interfere with the key blocks when the team is about to finish stacking". Another 8 subjects believe that the AI's ability to predict the target and control the timing of interference is very accurate, as stated in P3, P7, and P10: "The AI can quickly locate my key stacking blocks, forcing me to mislead through false movements, making the game no longer just stacking, but with a deeper level of strategy".

AI intelligence overall: All participants agreed that Dueler broke the stereotype of "fixed-rule AI",

and the two-stage training and dynamic camouflage made the AI's behavior more flexible and human-like (P9: "This AI is not a rigid 'machine', it has its own 'strategy', which is very different from the block game AI I have played before").

3D interface and mode design: Most participants thought the 3D block stacking interface was intuitive, and the diversified mode design (single/multi/AI confrontation) met different play needs (P2, P5: "The switch between different modes is simple, and the 3D effect makes the block stacking more intuitive than 2D").

4.3.2 Negative Feedback

Racing mode vs Dueler: the AI's speed is fixed from start to end, which is 1 second and such time is too fast for a human; it means the player almost cannot win Dueler in racing mode.

Confrontation mode vs Dueler: 2 participants thought the AI's camouflage probability adjustment was not perfect, and the camouflage behavior was slightly obvious in the mid-game (P3, P7: "In the middle of the game, the AI's fake moves are a bit repetitive, and it is easy to see that it is hiding the real target").3 participants suggested increasing the randomness of AI interference (P5, P9, P10: "Sometimes the AI only interferes with a certain type of block, which is easy to find the rule and avoid it"), in addition, occasionally there are erroneous blocks moves, such as moving a module that is behind the other blocks.

General operation and interface problems (cross-mode):4 participants suggested that although there is a 3D perspective adjustment function but because the label of block is at the front of it, which makes it difficult to see what the label is, when it is hidden by the other blocks.

Game richness: Most participants suggested adding more game elements, such as sound effects, animation feedback for block stacking, and a score/ranking system (P1-P10 all mentioned this point).

4.3.3 Software Improvement Plan Based on User Feedback

Due to the time limit of the project development cycle, the above problems and suggestions have not been implemented in the current version of the software. However, based on the user feedback (especially the feedback on the vs Dueler modes), the following specific and feasible improvement plans are proposed for future iteration.

For the Racing mode AI:

Add multi-level AI speed adjustment function (easy/medium/hard) to solve the problem that the fixed 1-second movement speed is too fast for human players. The speed settings are easy (2 seconds/step), medium (1.5 seconds/step), hard (1 second/step).

Introduce dynamic speed adaptation mechanism. The AI's initial movement speed is set to 1.5 seconds/step, and it will adjust according to the player's progress. if the player's completion progress is 30% higher than the AI's, the AI's speed will increase by 0.2 seconds/step; if the player's progress is 30% lower than the AI's, the AI's speed will decrease by 0.2 seconds/step.

For the Confrontation mode AI:

Optimize the dynamic camouflage strategy to solve the problem of repetitive fake moves. Expand the camouflage action library to include multiple types of deceptive behaviors (e.g., moving blocks to edge areas, swapping positions of non-target blocks, temporarily placing blocks near the player's stack and then moving them away). The camouflage probability curve is adjusted to a non-linear distribution, and the type of fake moves is randomly selected in each round to avoid predictability.

Expand the camouflage action library to include multiple types of deceptive behaviors (e.g., moving blocks to edge areas, swapping positions of non-target blocks, temporarily placing blocks near the player's stack and then moving them away). The camouflage probability curve is adjusted to a non-linear distribution, and the type of fake moves is randomly selected in each round to avoid predictability.

Expand the interference target range that instead of focusing on a single type of block, the AI will randomly select key blocks, surrounding auxiliary blocks, or even temporarily occupy the player's target stacking position according to the predicted player's goal.

Finally, before executing a movement, the AI will first verify whether the target block is the top block of its stack or has no overlapping/obstructing blocks in the movement path. If the block is inaccessible, the AI will automatically switch to an alternative interference or camouflage action, ensuring all operations comply with the game's physical rules.

In addition, it would also take some general optimization like implement a "label always visible" function that block labels (A/B/C/D/E/F) will be rendered in a semi-transparent floating layer that is always on top of the 3D scene to solve the problem of obscured labels, and add the multi-dimensional sound effects and the global score and ranking system.

5 Project Ethics

This project is classified as Data Category B and Participant Category 2, and I hereby confirm that I have read and strictly adhered to the university's ethical guidelines throughout the entire project. No activities were undertaken outside the scope of the approved collective ethics approval, no ethical issues or adverse events arose during the research, and all work complies with the requirements of academic integrity and professional codes of practice.

All data used for AI model training in this project is fully synthetic, generated via custom in-house Python scripts, with no scraping of external platforms or use of identifiable personal data.

In the aspect of software usability evaluation, ten adult subjects with full civil capacity were recruited in this study, and all subjects had signed informed consent. Before the formal test, each subject completely read the Subject Information Notification Form, which clearly explained the purpose of the study, the test process, the scope of data collection, and the relevant rights of the subject to unconditionally withdraw from the study at any time without bearing any adverse consequences. Every participant signed a written informed consent form before the testing session, confirming their voluntary participation and consent to the collection and use of their anonymized data. No financial incentives were offered to participants, in strict alignment with the department's ethical guidance.

All data collected from user testing is fully anonymized, with participants identified only by randomized codes (P1-P10) with no link to their personal identities. No personal identifiable information (PII) was collected, stored or processed at any stage of the project. All raw participant data was stored exclusively on a password-protected university-managed file store, accessible only to the project researcher, and will be permanently deleted 2 years after project completion. Only aggregated, fully anonymized results are included in this dissertation, with no individual data that could inadvertently identify any participant.

6 Conclusion and further work

6.1 Conclusion

This project aims to address the core limitation of traditional Blocks World AI, which relies on rigid

fixed rules and lacks adaptive intention perception and strategic confrontation capabilities in multi-player shared-space adversarial scenarios. I have successfully designed and implemented Blocks World with AI-Dueler, a 3D block stacking game integrated with an intelligent adversarial agent, and fully delivered all the core functional and research objectives proposed in the introduction.

In terms of system implementation, I built a complete three-tier architecture of "front-end application layer + back-end service layer + AI model layer". The front-end, based on Vue2 and Three.js, realized smooth 3D interaction and multi-mode scene switching; the back-end, built with Python Flask, completed game state management, multiplayer online functionality and AI model scheduling. The system fully supports four core gameplay modes, with a 100% pass rate in functional usability testing.

For the core AI agent, I proposed an innovative "two-stage training + dynamic camouflage" architecture. In racing mode, the designed heuristics achieve operational efficiency comparable to that of ordinary human players. In the shared space adversarial mode, the Transformer-based target prediction model achieves 98.9%verification accuracy, and the success rate of the PPO-based adversarial strategy model to prevent players from completing the task is 94%. The dynamic camouflage strategy effectively solves the single defect of traditional fixed rule AI behavior pattern, and the adversarial model for Dueler is highly recognized by the participants.

This project constructs an integrated agent framework of "target prediction, strategy confrontation, dynamic camouflage", which fills the research gap of adversarial AI in Blocks World, and has important academic reference and practical application significance.

6.2 Future Work

Future research work will focus on four directions to optimize and:

First, it focuses on improving the generalization ability of the target prediction model, balancing the interference effect of the confrontation strategy model and its own goal completion efficiency, and adding the level difficulty adaptive mechanism to realize the all-round improvement of AI performance.

Secondly, the fluency and convenience of 3D interactive operation were optimized by combining

user test feedback. Added sound effects, animation feedback and leaderboard.

Thirdly, a lightweight database was introduced to achieve persistent storage of game states and optimize the interaction performance of the front-end and back-end to reduce latency. Promote cross-platform adaptation of mobile terminals.

Fourth, the multi-agent team confrontation mode was explored to enrich the complexity of the confrontation scene. The application of the platform in programming education, cognitive science research and other fields is expanded.

7 BCS and Self-Reflection

7.1 Compliance with BCS

7.1.1 Application of Practical and Analytical Skills

This project comprehensively uses the practical operation and logical analysis ability acquired in the study. In terms of technical implementation, the project uses Vue2, Three.js 3D, Flask, and relies on PyTorch and Ray RLlib to complete the training of AI models. In terms of analysis and design, this paper summarizes the shortcomings of traditional Blocks World AI, and proposes a solution of two-stage training combined with dynamic camouflage for core problems such as goal reasoning and strategy confrontation. In addition, the project uses data analysis in model performance testing and user feedback evaluation to ensure the objectivity of the results.

7.1.2 Innovation and Creativity

The core innovation lies in new play mode in block game. The proposed "two-stage training + dynamic camouflage" architecture integrates target prediction, strategic decision-making, and behavior concealment, upgrading human-computer confrontation from "puzzle racing" to a "strategy game". This fills the research gap in goal reasoning and strategy camouflage in Blocks World’s shared space mode.

7.1.3 Synthesis of Information and Solution Evaluation

The project synthesizes multi-disciplinary knowledge (3D development, backend services, machine learning, usability testing) to form a complete solution. It combines heuristic algorithms and deep reinforcement learning for different game modes, and verifies effectiveness through three-

dimensional evaluation: functional usability testing, model performance testing, and user experience evaluation.

7.1.4 Meeting Real Needs in a Wider Context

Academically, it provides an experimental platform for AI adversarial algorithm verification in Blocks World, verifying the effectiveness of deep learning and reinforcement learning combination in intention reasoning. Practically, it solves the single gameplay issue of traditional block games, providing users with a strategic 3D stacking experience. The "target prediction - strategy confrontation - dynamic camouflage" framework can be extended to other puzzle competitive games.

7.1.5 Ability to Self-Manage a Significant Piece of Work

As an independent research work, this project adopts the promotion method of phased time control and modular task splitting with Gantt.

7.2 Self-Evaluation

Through the practice of this project, I realized all-round growth, and clearly realized my own shortcomings:

Strengths

Strictly follow the "requirements analysis → design → development and test" process, complete the tasks on time, and successfully achieve all the preset indicators.

In the face of technical problems such as 3D collision detection and model overfitting, it can actively seek solutions by consulting literature.

User feedback is collected through usability testing, and targeted optimization schemes are proposed to ensure that the product meets user needs.

Shortcomings

The target prediction model has weak generalization ability for unseen targets. The adversarial model focuses too much on the "interference" function, which leads to the limited adaptation of the model.

Problems such as cross-browser 3D rendering compatibility and long model training time caused

by hardware performance limitations are not fully predicted.

The lack of modularity and low unit test coverage affect the maintainability and robustness of the code.

Improvement Directions

The open vocabulary model and multi-objective reinforcement learning related technologies should be studied to optimize the generalization ability of the target prediction model and improve the scientific strategy of the confrontation model.

More detailed risk assessments should be carried out, and project management tools should be introduced to track project progress in real time.

We should add code comments, strengthen module splitting, expand unit test coverage, and improve code maintainability.

This project not only completed the established system development task, but also clarified its own advantages and disadvantages through self-reflection, which accumulated valuable practical experience for subsequent study and work.

References

1. Axios. 2024. Axios HTTP Client Documentation. [Online]. Available at: https://axios-http.com/docs/intro. Accessed: 15 Jan 2026.
2. Eleme. 2024. Element UI 2.x Official Documentation. [Online]. Available at: https://element.eleme.io/#/en-US. Accessed: 12 Jan 2026.
3. Fielding, R.T. 2000. Architectural styles and the design of network-based software architectures. PhD dissertation, University of California, Irvine. DOI: 10.5555/932639. Available at: https://roy.gbiv.com/pubs/dissertation/top.htm. Accessed: 18 Feb 2026.
4. Hart, P.E., Nilsson, N.J. & Raphael, B. 1968. A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), pp.100-107. DOI: 10.1109/TSSC.1968.300136. Available at: https://ieeexplore.ieee.org/document/4082128. Accessed: 20 Jan 2026.
5. Korf, R.E. 1985. Depth-first iterative-deepening: an optimal admissible tree search. Artificial Intelligence, 27(1), pp.97-109. DOI: 10.1016/0004-3702(85)90084-0. Available at: https://www.sciencedirect.com/science/article/pii/0004370285900840. Accessed: 22 Jan 2026.
6. Leach, P., Mealling, M. & Salz, R. 2005. A Universally Unique IDentifier (UUID) URN Namespace. IETF RFC 4122. [Online]. Available at: https://datatracker.ietf.org/doc/html/rfc4122. Accessed: 18 Feb 2026.
7. Liang, E., Liaw, R., Nishihara, R., Moritz, P., Fox, R., Goldberg, K., Gonzalez, J., Jordan, M.I. & Stoica, I. 2018. RLlib: Abstractions for distributed reinforcement learning. Proceedings of the 35th International Conference on Machine Learning (ICML), Stockholm, Sweden, pp.3053-3062. Available at: https://proceedings.mlr.press/v80/liang18b.html. Accessed: 25 Jan 2026.
8. Lim, B., Arık, S.Ö., Loeff, N. & Pfister, T. 2021. Temporal fusion transformers for interpretable multi-horizon time series forecasting. International Journal of Forecasting, 37(4), pp.1748-1764. DOI: 10.1016/j.ijforecast.2021.03.012. Available at: https://www.sciencedirect.com/science/article/pii/S0169207021000637. Accessed: 22 Jan 2026.
9. Loshchilov, I. & Hutter, F. 2017. SGDR: Stochastic gradient descent with warm restarts.

Proceedings of the 5th International Conference on Learning Representations (ICLR), Toulon, France. Available at: https://openreview.net/forum?id=Skq89Scxx. Accessed: 25 Jan 2026.

10. Loshchilov, I. & Hutter, F. 2019. Decoupled weight decay regularization. Proceedings of the 7th International Conference on Learning Representations (ICLR), New Orleans, USA. Available at: https://openreview.net/forum?id=Bkg6RiCqY7. Accessed: 25 Jan 2026.
11. Lowe, R., Wu, Y.I., Tamar, A., Harb, J., Abbeel, P. & Mordatch, I. 2017. Multi-agent actor-critic for mixed cooperative-competitive environments. Advances in Neural Information Processing Systems, 30. DOI: 10.48550/arXiv.1706.02275. Available at: https://arxiv.org/pdf/1706.02275. Accessed: 20 Jan 2026.
12. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G. & Petersen, S. 2015. Human-level control through deep reinforcement learning. Nature, 518(7540), pp.529-533. DOI: 10.1038/nature14236. Available at: https://www.nature.com/articles/nature14236. Accessed: 20 Jan 2026.
13. Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M.I. & Stoica, I. 2018. Ray: A distributed framework for emerging AI applications. 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI), Carlsbad, USA, pp.561-577. Available at: https://www.usenix.org/conference/osdi18/presentation/moritz. Accessed: 25 Jan 2026.
14. Nilsson, N.J. 1980. Principles of artificial intelligence. Tioga Publishing Company. ISBN: 0935382011. Available at: https://www.sciencedirect.com/book/monograph/9780934613101/principles-of-artificial-intelligence. Accessed: 15 Jan 2026.
15. Pallets Projects. 2024. Flask 2.0.x Official Documentation. [Online]. Available at: https://flask.palletsprojects.com/en/2.0.x/. Accessed: 10 Jan 2026.
16. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L. & Desmaison, A. 2019. PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32. Available at: https://proceedings.neurips.cc/paper_files/paper/2019/hash/bdbca288fee7f92f2bfa9f7012

727740-Abstract.html. Accessed: 20 Jan 2026.

17. Schulman, J., Levine, S., Abbeel, P., Jordan, M. & Moritz, P. 2015. Trust region policy optimization. Proceedings of the 32nd International Conference on Machine Learning (ICML), Lille, France, pp.1889-1897. Available at: https://proceedings.mlr.press/v37/schulman15.html. Accessed: 20 Jan 2026.
18. Schulman, J., Moritz, P., Levine, S., Jordan, M. & Abbeel, P. 2016. High-dimensional continuous control using generalized advantage estimation. Proceedings of the 4th International Conference on Learning Representations (ICLR), San Juan, Puerto Rico. Available at: https://openreview.net/forum?id=l8ZdsoyUEYM. Accessed: 25 Jan 2026.
19. Schulman, J., Wolski, F., Dhariwal, P., Radford, A. & Klimov, O. 2017. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347. DOI: 10.48550/arXiv.1707.06347. Available at: https://arxiv.org/abs/1707.06347. Accessed: 18 Jan 2026.
20. Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T. & Lillicrap, T. 2018. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. Science, 362(6419), pp.1140-1144. DOI: 10.1126/science.aar6404. Available at: https://www.science.org/doi/10.1126/science.aar6404. Accessed: 22 Jan 2026.
21. Slaney, J. & Thiébaux, S. 2001. Blocks world revisited. Artificial Intelligence, 125(1-2), pp.119-153. DOI: 10.1016/S0004-3702(00)00079-5. Available at: https://www.sciencedirect.com/science/article/pii/S0004370200000795. Accessed: 15 Jan 2026.
22. Sutton, R.S. & Barto, A.G. 2018. Reinforcement learning: An introduction. 2nd ed. MIT Press. ISBN: 9780262039240. Available at: https://blog.csdn.net/weixin_44909139/article/details/127175232. Accessed: 10 Jan 2026.
23. Three.js. 2024. Three.js r158 Official Documentation. [Online]. Available at: https://threejs.org/docs/index.html#manual/en/introduction/Creating-a-scene. Accessed: 12 Jan 2026.
24. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. & Polosukhin, I. 2017. Attention is all you need. Advances in Neural Information Processing Systems, 30. DOI: 10.48550/arXiv.1706.03762. Available at:

https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html. Accessed: 18 Jan 2026.

25. Vue.js. 2024. Vue 2.x Official Documentation. [Online]. Available at: https://v2.vuejs.org/v2/guide/. Accessed: 10 Jan 2026.
26. Vue.js. 2024a. Vue Router 3.x Official Documentation. [Online]. Available at: https://v3.router.vuejs.org/guide/. Accessed: 12 Jan 2026.
27. Yu, C., Velu, A., Vinitsky, E., Wang, Y., Bayen, A. & Wu, Y. 2021. The surprising effectiveness of PPO in cooperative, multi-agent games. arXiv preprint arXiv:2103.01955. DOI: 10.48550/arXiv.2103.01955. Available at: https://arxiv.org/abs/2103.01955. Accessed: 20 Jan 2026.

Appendix

Appendix 1. Storyboard

Storyboard 1: Single game

Order	Scenario	Description	User operating
1	Open the game	The user launches the Blocks World game in the browser, and the welcome page (with mode selection and configuration entry) is displayed.	Click the game icon/link to open the game in the browser.
2	Enter configuration page	The user needs to set the player name and block quantity before starting the single-player mode.	Click the "Configuration" button on the welcome page to enter the configuration interface.
3	Configure player information	The configuration page provides a name input box and a block quantity selector (5/6/7 options).	Enter "player1" in the name input box; select "5" from the block amount dropdown menu.
4	Save configuration and return	After completing the configuration, the user needs to confirm the settings to return to the welcome page.	Click the "Save & Return" button on the configuration page.
5	Select single-player mode	The welcome page displays multiple game modes, and the user chooses the single-player training mode.	Click the "Single-Player" button on the welcome page.
6	Stack blocks according to the target	The user drags the top blocks one by one, adjusting their positions to match the target stack. Invalid operations (e.g., moving non-top blocks) will trigger a pop-up prompt.	Drag the top blocks sequentially with the mouse to complete the stack.

7	Complete the target and get prompted	After the user's block stack fully matches the target, the system automatically judges victory and pops up a success prompt.	Wait for the victory prompt to appear; no additional operation is required during judgment.
8	Return to the welcome page	The success prompt provides a "Return to Welcome Page" option to allow the user to restart or switch modes.	Click the "Return to Welcome Page" button in the prompt box.

Storyboard 2: Multiplayer racing mode

Order	Scenario	Description	User operating
1	Player 1 opens the game and selects race mode	Player 1 launches the game, enters the welcome page, and intends to create a multiplayer racing room.	Open the game in the browser; click the "Race Mode" button on the welcome page.
2	Player 1 chooses "VS Player" and creates a host	The race mode interface provides two options: "VS Player" and "VS Dueler". Player 1 selects to play against another player and creates a game room.	Click the "VS Player" option; then click the "Create Host" button, and the system generates a unique room ID.
3	Player 1 waits for other players to join	The game displays a "Waiting for another player" prompt. Player 1 stays in the room interface.	Keep the game page open and wait for Player 2 to join.
4	Player 2 opens the game and enters the game hall	Player 2 launches the game, enters the welcome page, and selects the race mode to join an existing room.	Open the game in the browser; click the "Race Mode" button, then click the "Join Room" button to enter the game hall.
5	Player 2 joins the host room	The game hall displays all active rooms. Player 2 finds Player 1's	Click the "Join" button next to the target room.

		room by room ID or room name.	
6	Both players enter the independent racing interface	After Player 2 joins successfully, the system loads two independent 3D scenes (one for each player) with the same initial block state and target stack "ABCED".	Automatically enter the game interface and start game.
7	Start multiplayer racing	The countdown ends, and both players start dragging blocks to stack toward the target "ABCED". The system synchronizes the step count and progress in real time.	Drag the top blocks with the mouse to complete the target stack as quickly as possible.
8	Determine the winner and display results	The first player whose stack fully matches "ABCED" triggers the victory judgment. The system pops up a result prompt for both players.	The winning player views the "Victory" prompt; the losing player views the "Defeat" prompt.
9	Exit the game	The result prompt provides "Return to Welcome Page" options.	Both players click the desired button (e.g., "Return to Welcome Page") to end the session.

Storyboard 3: Man-machine racing mode

Order	Scenario	Description	User operating
1	Open the game and enter race mode	The user launches the game, enters the welcome page, and selects the race mode to compete against the AI (Dueler).	Open the game in the browser; click the "Race Mode" button on the welcome page.
2	Select "VS Dueler" and confirm	The user chooses to play against Dueler, and the system loads the configured block quantity (5 blocks,	Click the "VS Dueler" option; confirm the block amount is "5" and click "Next".

	settings	consistent with the default or pre-configured settings).	
3	Enter the human-AI racing interface	The system displays two side-by-side 3D scenes: the left for the user and the right for Dueler. Both share the target stack "ABCED".	
4	Dueler performs automatic movement	Dueler uses a heuristic algorithm, executing a movement every 2 seconds to approach the target stack efficiently.	Observe Dueler's real-time movement in the right scene; prepare for own operation.
5	User operates to stack blocks	The user drags blocks to stack toward "ABCED", but the operation speed is slower than Dueler's.	Drag the top blocks with the mouse to adjust their positions; correct invalid operations based on system prompts.
6	Dueler completes the target first	Dueler's stack fully matches "ABCED" before the user, and the system immediately triggers the victory judgment for Dueler.	The user's operation is interrupted by the "Dueler Wins" pop-up prompt.
7	View result and exit	The result prompt displays Dueler's completion with "Return to Welcome Page" button.	Click the "Return to Welcome Page" button.

Storyboard 4: Man-machine confrontation mode

Order	Scenario	Description	User operating
1	Open the game and select confrontation mode	The user launches the game, enters the welcome page, and chooses the shared space confrontation mode to play against Dueler.	Open the game in the browser; click the "Confrontation Mode" button on the welcome page.

2	Select "VS Dueler" and confirm block quantity	The user selects to compete against Dueler, and the system confirms the block amount is 5 (matching the target "ABCED").	Click the "VS Dueler" option
3	Enter shared space 3D scene	The game loads a single 3D scene (shared tabletop + 5 blocks A/B/C/D/E). The user's target "ABCED" is displayed, while Dueler's target is hidden. Both take turns to operate.	Wait for the "Player 1's Turn" prompt to start the game.
4	Player's first round (fake action)	To deceive Dueler, the user performs a fake operation (e.g., drag block A to a random position and immediately drag it back) without approaching their own target.	Drag block A from its initial position to an irrelevant area with the mouse, then drag it back to the original stack.
5	Dueler's round (fake action/interference)	Dueler's turn: based on the user's fake action, it cannot accurately predict the target and chooses to perform a fake operation (drag block B back and forth).	Observe Dueler's movement; wait for the "Player 1's Turn" prompt to reappear.
6	Alternate turns (fake actions + gradual stacking)	The user alternates between fake actions and real stacking: in subsequent rounds, the user subtly adjusts key blocks (e.g., place E on the table, D on E) while occasionally performing fake moves. Dueler attempts to interfere but fails due to incorrect target prediction.	Drag block E to the center of the table (real action); next round, drag block C to a random position and back (fake action); then drag D to E (real action).

7	Dueler’s interference fails	Dueler predicts the user’s target incorrectly (e.g., assumes the target is "BCAED") and interferes with block C, which does not affect the user’s real target "ABCED".	Ignore Dueler’s interference; continue to stack B on D, A on B.
8	Player completes the target stack	The user finally places block A on B, forming the target stack "A→B→C→D→E→table". The system judges the user’s victory.	Confirm the stack matches the target; wait for the victory prompt.
9	View result and exit	The pop-up prompt displays "Victory! You deceived Dueler and completed the target" with options to "Play Again" or "Return to Welcome Page".	Click the "Return to Welcome Page" button to end the game.

Appendix 2. Non-Interference Data Generation Output

=== Data Statistics ===

Total valid records: 100000

ABCDEF target permutation distribution:

ABCDEF: 3406 samples (3.41%)

FEDCBA: 3272 samples (3.27%)

ABCDFE: 3239 samples (3.24%)

ABCEDF: 3312 samples (3.31%)

ABCFED: 3359 samples (3.36%)

ABDCEF: 3365 samples (3.36%)

ACBDEF: 3273 samples (3.27%)

ABDEFC: 3321 samples (3.32%)

BACDEF: 3258 samples (3.26%)

BACEDF: 3283 samples (3.28%)

CBAEDF: 3259 samples (3.26%)

CBADEF: 3330 samples (3.33%)

CDABEF: 3419 samples (3.42%)

DABCEF: 3286 samples (3.29%)

DBFACE: 3409 samples (3.41%)

EABCDF: 3232 samples (3.23%)

EBACDF: 3334 samples (3.33%)

ECABDF: 3304 samples (3.30%)

EDABCF: 3338 samples (3.34%)

DEABCF: 3387 samples (3.39%)

FABCDE: 3393 samples (3.39%)

FBACED: 3402 samples (3.40%)

FCABDE: 3318 samples (3.32%)

FDABCE: 3351 samples (3.35%)

FEABCD: 3320 samples (3.32%)

ABFCDE: 3401 samples (3.40%)

ACFBDE: 3386 samples (3.39%)

ADFBCE: 3313 samples (3.31%)

AEFBCD: 3360 samples (3.36%)

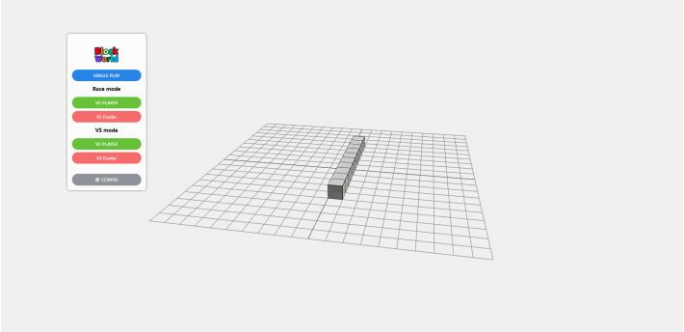
Appendix 3. Reward weights

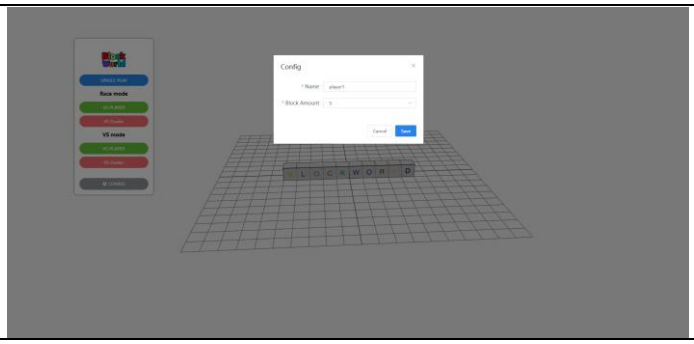
Reward Component	**Value**	**Description**
Prevent player victory	+100	Successfully preventing player from completing their goal
AI completes goal	+80	Successfully achieving AI's own goal
Player completes goal	-150	Penalty when player succeeds (mission failure)

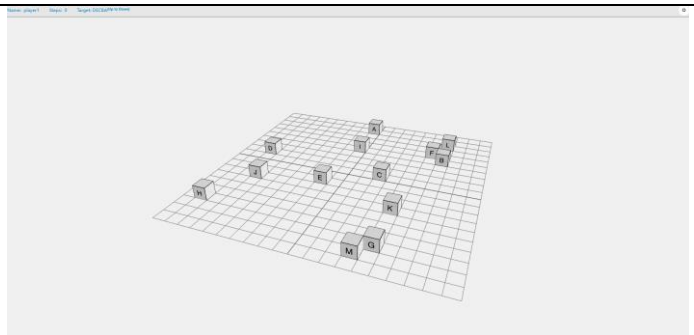
Correct timing interference	+50	Interfering when player has 4-5 blocks stacked (optimal timing)
Wrong timing interference	-30	Penalty for interfering too early (< 4 blocks) or too late (> 5 blocks)
Effective interference	+15	Successfully reducing player's progress
AI progress	+5 per step	Reward for making progress toward AI's own goal
Step penalty	-0.1 per step	Encourages efficient completion
Progress maintenance	+0.5 × (6 - player_progress)	Continuous reward for keeping player progress low

Appendix 4. Test based on storyboard

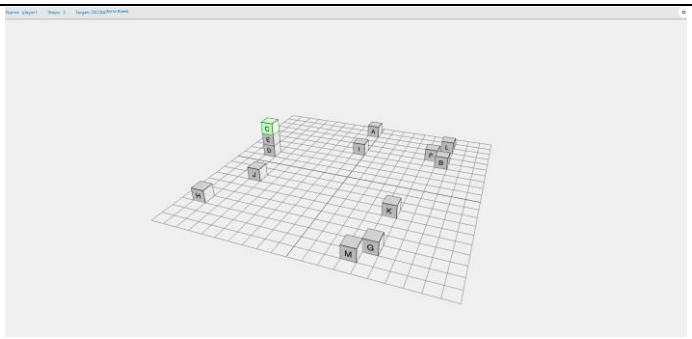
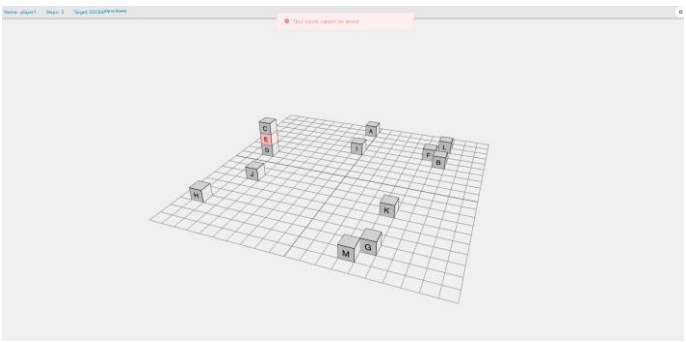
Single Game

Function Number	1
Function Description	Open the game
Operating	Click the game link
Expected	Successful open the game in the browser
Actual	Successful open the game in the browser
Screenshot	
Test Result	Pass

Function Number	2
Function Description	Set configuration
Operating	Click the "Configuration" button, Enter "player1" in the name input box; select "5" and click save
Expected	Successful save configuration
Actual	Successful save configuration r
Screenshot	
Test Result	Pass

Function Number	3
Function Description	Start single-player game
Operating	Click the "Single-Player" button, start the game.
Expected	Successful enter and display the single play game and information including user name, target, steps.
Actual	Successful enter and display the single play game and information including user name, target, steps.
Screenshot	
Test Result	Pass

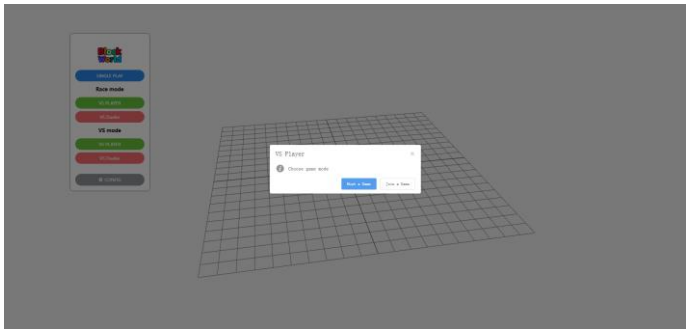
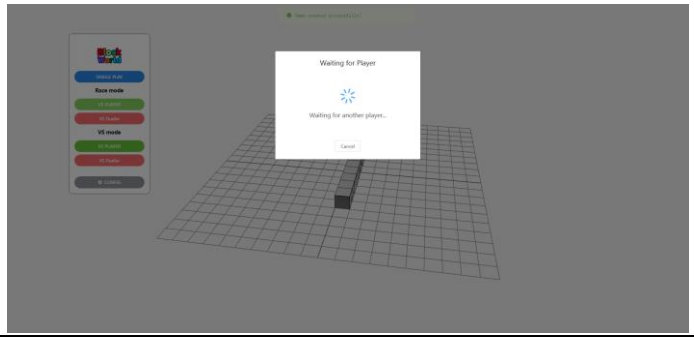
Function Number	4

Function Description	Stack blocks according to the target
Operating	Drags the top blocks one by one to match the target stack.
Expected	Successful place the valid block and reject to place if the block position is illegal.
Actual	Successful place the valid block and reject to place if the block position is illegal.
Screenshot	 
Test Result	Pass

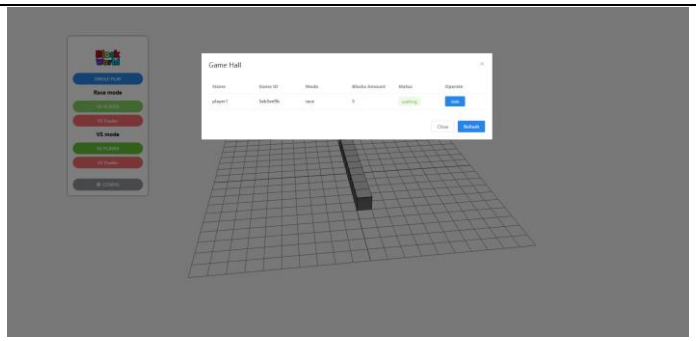
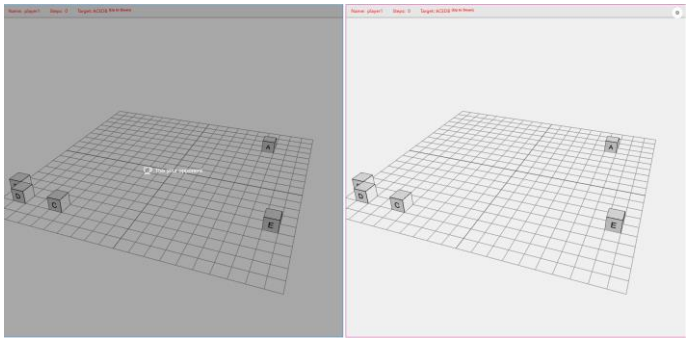
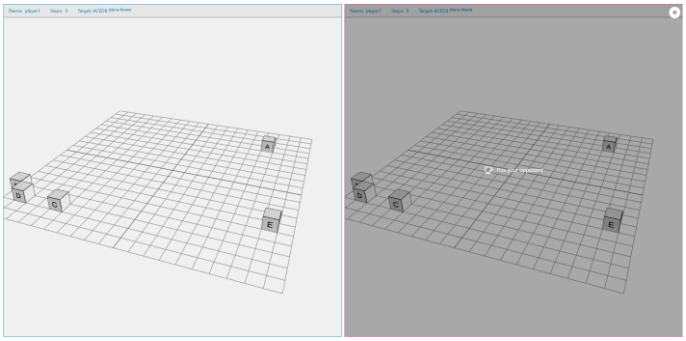
Function Number	5
Function Description	Complete the target and get prompted
Operating	Drags the blocks to complete target and click the dialog to back to home
Expected	Successful see the win dialog and return home
Actual	Successful see the win dialog and return home

Screenshot	
Test Result	Pass

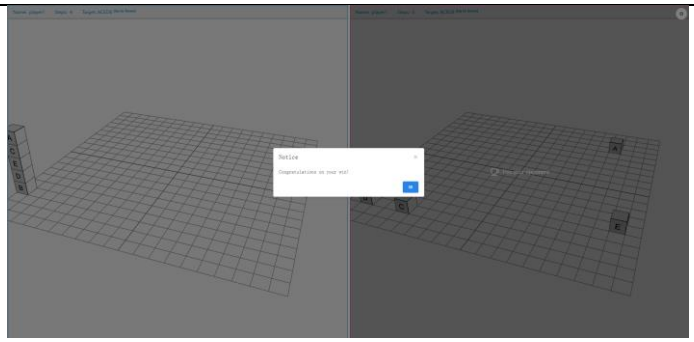
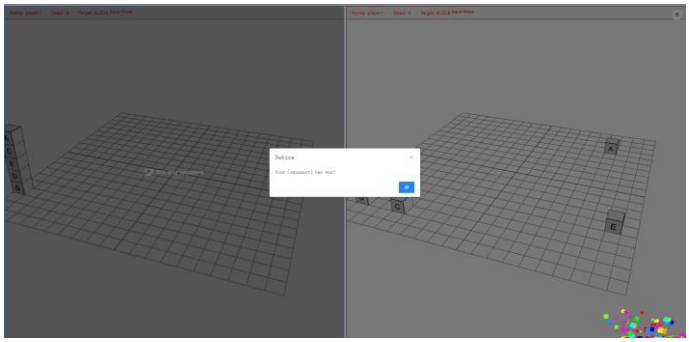
Multiplayer racing mode

Function Number	1
Function Description	Player 1 chooses race mode "VS Player" and creates a host
Operating	Click the "VS Player" option in race mode and click the "Create Host" button
Expected	Successful enter waiting status for a host
Actual	Successful enter waiting status for a host
Screenshot	 
Test Result	Pass

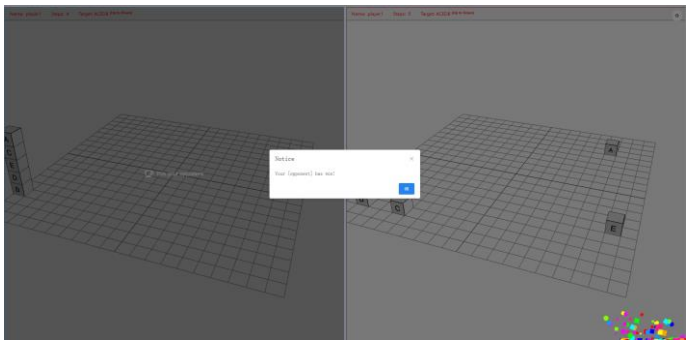
Function Number	2
Function Description	Player 2 joins the game

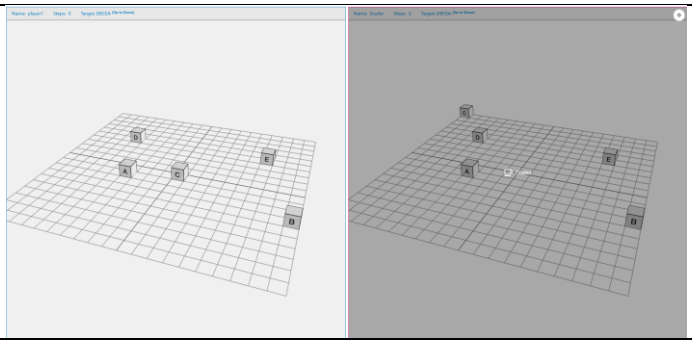
Operating	Player 2 finds Player 1’s room and click the "Join" button
Expected	Successful enter race mode game and start game
Actual	Successful enter race mode game and start game
Screenshot	  
Test Result	Pass

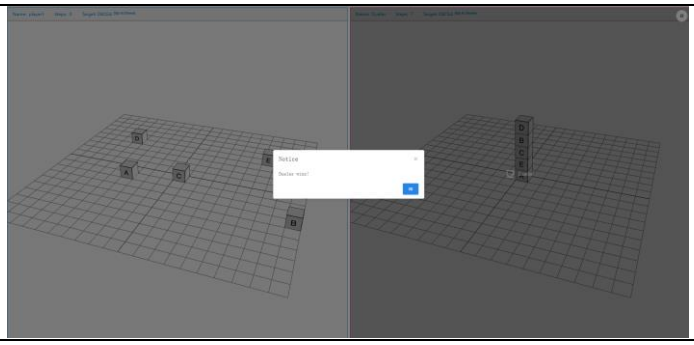
Function Number	3
Function Description	Player 1 completes the game at first
Operating	Player 1 completes the target at first to see win dialog
Expected	Player 1 sees win dialog, Player 2 see defeat dialog
Actual	Player 1 sees win dialog, Player 2 see defeat dialog

Screenshot	 
Test Result	Pass

Man-machine racing mode

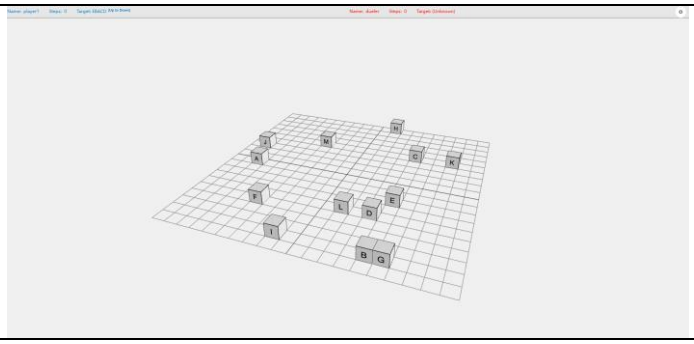
Function Number	1
Function Description	Player 1 selects the race mode to compete against the AI
Operating	Player 1 Click the "VS Dueler" option in race mode
Expected	Player 1 enter race mode and game starts directly
Actual	Player 1 enter race mode and game starts directly
Screenshot	
Test Result	Pass

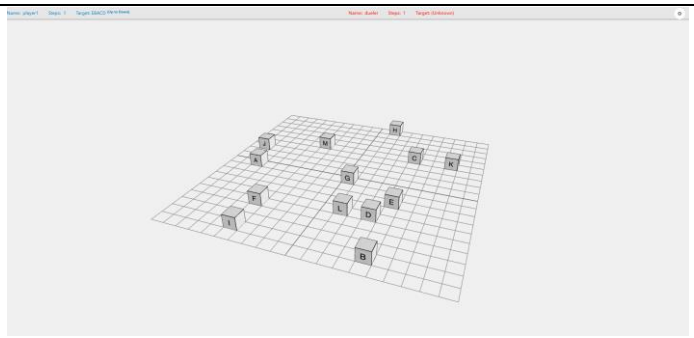
Function Number	2
Function Description	Player 1 selects the race mode to compete against the AI
Operating	Player 1 Click the "VS Dueler" option in race mode
Expected	Player 1 enter race mode and game starts directly
Actual	Player 1 enter race mode and game starts directly
Screenshot	
Test Result	Pass

Function Number	3
Function Description	Player 1 no movement, Dueler win the game
Operating	No operation waiting Dueler win
Expected	Dueler moves block in reasonable method and win game
Actual	Dueler moves block in reasonable method and win game
Screenshot	
Test Result	Pass

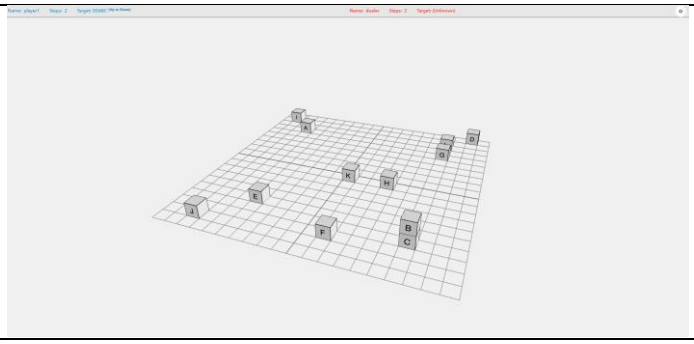
Man-machine confrontation mode

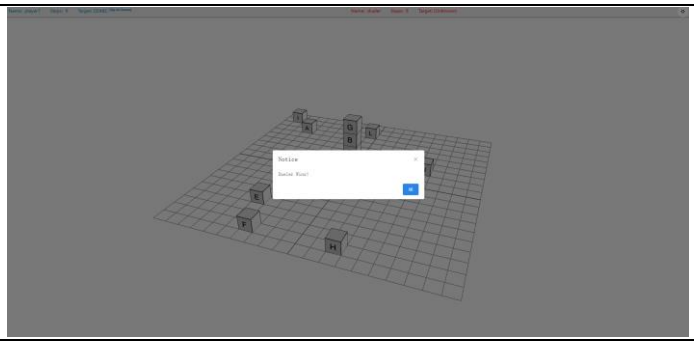
Function Number	1
Function Description	Player 1 selects the confrontation mode to compete against the AI
Operating	Player 1 Click the "VS Dueler" option in confrontation mode

Expected	Player 1 enter confrontation mode and game starts directly
Actual	Player 1 enter confrontation mode and game starts directly
Screenshot	
Test Result	Pass

Function Number	2
Function Description	AI's Misleading behavior
Operating	No operation
Expected	AI deliberately moving blocks unrelated to the target misleads the player
Actual	AI deliberately moving blocks unrelated to the target misleads the player
Screenshot	
Test Result	Pass

Function Number	3
Function Description	AI's interference
Operating	Player 1 stack 3-4 blocks and is terminated by AI
Expected	Player 1 one key block is moved away by AI (A is moved away)
Actual	Player 1 one key block is moved away by AI (A is moved away)

Screenshot	
Test Result	Pass

Function Number	4
Function Description	AI’s win
Operating	Player 1 move blocks randomly
Expected	AI win the game
Actual	AI win the game
Screenshot	
Test Result	Pass

Appendix 5. Test of accuracy of predicted target

Test 1: Memory Capability (Training Data) ● Top-1 Accuracy: 98.9% (89/90 samples) ● Top-3 Accuracy: 100.0% (90/90 samples) ● Top-5 Accuracy: 100.0% (90/90 samples) This indicates the model successfully learned the training distribution with minimal overfitting.
Test 2: Same-Target Generalization (Varied Initial States) ● Top-1 Accuracy: 100.0% (30/30 samples)

• Top-3 Accuracy: 100.0% (30/30 samples) • Top-5 Accuracy: 100.0% (30/30 samples) This demonstrates robust feature extraction that focuses on block relationships rather than absolute positions.
Test 3: Novel-Target Generalization (Unseen Permutations) • Top-1 Accuracy: 0.0% (0/15 samples) • Top-3 Accuracy: 0.0% (0/15 samples) • Top-5 Accuracy: 0.0% (0/15 samples) For all 5 unseen targets (DEFABC, CDEFAB, BCDEAF, ACDEFB, BCDEFA), the model consistently predicted trained permutations with 100% confidence, often defaulting to similar patterns (e.g., ABCDEF for DEFABC). This behavior indicates the model operates as a classifier rather than a compositional reasoner.

Appendix 6. Participant Consent Form

Project Title: Usability Testing of "Blocks World with AI-Dueler" Game

Researcher: [Researcher's Name/Team Name]

1. Purpose of the Research

This research aims to evaluate the usability, playability, and AI interaction experience of the 3D block stacking game "Blocks World with AI-Dueler", including single-player mode, multiplayer racing mode, and human-AI confrontation mode (vs Dueler). Your participation will help us identify potential problems in the game and optimize its functions and user experience.

2. What Your Participation Involves

- You will be asked to complete a 40-50 minute game experience session, including trying all core game modes (single-player, multiplayer racing, human-AI racing vs Dueler, human-AI confrontation vs Dueler).
- After the experience, you will fill out a structured questionnaire (5-10 minutes) and participate in a short semi-structured interview (3 minutes) to share your feelings and

suggestions.

- The researcher will record objective data during your game (e.g., operation duration, success/failure rate, number of invalid operations) but will not collect any personal sensitive information.

3. Voluntary Participation

- Your participation in this research is completely voluntary. You have the right to refuse to participate at any time without any negative impact.
- If you choose to participate, you can withdraw from the research at any stage, and we will not use any data collected before your withdrawal.

4. Data Collection and Usage

- All data collected (questionnaire responses, interview content, game operation data) will be anonymized. Your personal information (e.g., name) will not be associated with the research data, and all identifiers will be replaced with codes (e.g., P1, P2).
- The collected data will only be used for the optimization of the "Blocks World with AI-Dueler" game and related research reports. It will not be disclosed to third parties or used for commercial purposes.
- The data will be stored securely for 2 years after the completion of the research. After that, it will be permanently deleted or anonymized for long-term academic research (if applicable).

5. Risks and Benefits

- There are no known risks associated with this research. The game experience is non-invasive and will not cause physical or psychological harm.
- As a token of appreciation, you will receive a small gift (or digital reward) after completing the entire testing process. More importantly, your feedback will directly help improve the game’s quality and user experience.

6. Right to Ask Questions

- If you have any questions about this research, the game, or your rights as a participant, please feel free to ask the researcher before signing this form. The researcher will provide you with a detailed explanation.
- If you have additional questions after the research, you can contact the researcher via

[contact information] within 3 months of the testing date.

7. Consent

By signing below, you confirm that:

- You have read and understood the above information.
- You voluntarily agree to participate in this research.
- You agree to the collection and use of your anonymized data as described.
- You are 18 years of age or older (if you are under 18, a parent/guardian's signature is required below).

Participant's Signature: _____

Date: _____

Parent/Guardian's Signature (if participant is under 18): _____

Date: _____

Researcher's Signature: _____

Date: _____

Contact Information of Researcher: _____

Appendix 7. Feedback form

Question No.	**Question Content**	**Average Score (1-5)**
1	The game's 3D interface is clear and easy to understand	4.2
2	The block drag-and-drop operation is smooth and free of obvious delays	3.8
3	The single-player mode has a reasonable difficulty and good playability	4.1
4	The multiplayer racing mode has stable online synchronization	3.7
5	The AI (Dueler) in the game shows obvious intelligent decision-making ability	4.3
6	Your experience of playing against Dueler in the **racing mode**	4.0

	(smoothness, AI difficulty matching)	
7	Your experience of playing against Dueler in the **confrontation mode** (AI strategy, camouflage, interference fun)	4.4
8	The overall playability of the game meets your expectations	4.1

Appendix 8. Project structure

```
views/
├── config/                    # Configuration module
│   └── edit.vue               # Configuration edit page
├── error/                     # Error pages
│   ├── 403.vue                # 403 Permission denied page
│   └── 404.vue                # 404 Not found page
├── home/                      # single game page
│   └── index. vue             # single game page
├── main /                     # Main module
│   ├── index.vue              # welcome page
│   ├── room.vue               # Room page
│   └── service.js             # welcome module service
├── racemode/                  # Race mode
│   ├── vsdueler/              # Versus Dueler mode
│   │   ├── index.vue          # Dueler mode main page
│   │   ├── oneplayer.vue      # Single player page
│   │   └── service.js         # Dueler mode service
│   └── vsplayer/              # Versus Player mode
│       ├── index.vue          # Player mode main page
│       ├── oneplayer.vue      # Single player page
│       └── service.js         # Player mode service
├── redirect/                  # Redirect
│   └── index.vue              # Redirect page
└── vsmode/                    # VS mode
    ├── vsdueler/              # VS Dueler
    │   ├── index.vue          # Dueler main page
    │   ├── oneplayer.vue      # Single player page
    │   └── service.js         # Dueler service
    └── vsplayer/              # VS Player
        ├── index.vue          # Player main page
        └── oneplayer.vue      # Single player page
```